Simulation-informed transfer learning improves low-data nanobody thermal stability prediction

Taihei Murakami[†1], **Kentaro Sasaki**[†1], **and Yasuhiro Matsunaga**[*1,2]

[1]Saitama University, Saitama, Japan
[2]RIKEN Center for Computational Science, Kobe, Japan

[†]These authors contributed equally to this work.

Nanobody engineering depends on thermal stability, but experimental melting-temperature (Tm) measurements remain scarce. Computed stability labels can be generated more readily, yet they usually describe quantities other than Tm, such as mutation-level free-energy changes or structural scores. We asked whether such labels can improve low-data Tm prediction when they are used as computational tasks but models are selected only by experimental Tm performance. Using a public nanobody benchmark with 57 training, 114 validation, and 396 held-out test sequences, we trained multi-task models that shared an ESM2 sequence representation between a Tm target head and one computational task at a time. The computational labels included alchemical free-energy perturbation (FEP) mutation free energies, Rosetta and ThermoMPNN mutation scores, Rosetta scores for language-model-proposed variants, and molecular-dynamics (MD) trajectory summaries. FEP labels gave the strongest transfer, lowering held-out Tm mean absolute error from 6.61 to 6.26°C, a paired gain of 0.35°C (90% confidence interval, 0.24 to 0.46°C). This improvement was not reproduced by an MD-derived Q-value or by larger ESM2 encoders. Rosetta-scored proposal sets gave weaker gains that remained close to random-variant controls. These results show that simulation-derived labels can support scarce experimental Tm prediction, but only when the computational label carries physical information that transfers to the target phenotype.

nanobody thermostability | melting temperature | simulation-to-real transfer learning | protein language model | free energy perturbation

Correspondence: ***Yasuhiro Matsunaga, ymatsunaga@riken.jp***

Introduction

Machine learning can screen molecular designs before experimental testing, but its usefulness is limited by the labels available for training. In materials informatics, this bottleneck has motivated simulation-to-real (Sim2Real) learning, in which models are trained with large computational databases from first-principles calculations, quantum chemistry, or molecular dynamics, then adapted to smaller experimental data sets (1–3). In some polymer and inorganic-material settings, transfer error decreases as the computational database grows, providing a practical test of whether additional simulation is worth generating (3). That test is needed because simulation data are not automatically useful. Model assumptions, finite sampling, and domain gaps can separate a computed label from the experimental property it is meant to support.

Protein engineering has the same label bottleneck, but the relation between computational and experimental labels is often less direct. Experimental measurements are sparse, whereas simulations and structure-based calculations can label many designed or mutated sequences. Those computed labels, however, rarely measure the target phenotype itself. For thermal stability, the experimental target may be the melting temperature (Tm), the assay-dependent temperature at which folded and unfolded populations are balanced. The available computed quantity may instead be a mutation-level free-energy change ($\Delta\Delta G$), a structure-based stability score, or a molecular-dynamics trajectory summary. Such labels are physically related to stability, but they are not interchangeable with Tm. The central mismatch motivates a direct test of which computed quantities transfer to experimental Tm prediction.

We study this question for nanobodies, single-domain VHH antibodies that are small, modular, and useful as engineered binders (4, 5). Their practical use in expression, storage, and formulation depends on thermal stability (6–8), but measuring or simulating that stability directly is costly (9–12). Protein language models (PLMs) such as ESM2 provide a natural starting point because they learn sequence representations from large protein corpora (13–15), and several recent methods use such representations for Tm prediction (16–21). High-throughput stability assays are also changing the field. Tsuboyama et al. measured folding stability for roughly 776,000 natural and designed protein domains by cDNA-display proteolysis (22). Yet those data do not remove the need for nanobody-specific target supervision. The mega-scale domains are 40 to 72 residues long, whereas VHH domains are longer, and models fine-tuned on such data transfer less well to 177 to 501-residue proteins than to small domains (23). For nanobody Tm prediction, target-specific data and target-centered validation remain essential (24–26).

Here, we test whether computed stability labels can supply useful supervision when experimental nanobody Tm labels are scarce. We treat Tm prediction as the target task and each computed label as a computational task that shares an ESM2-based sequence representation (14). The computational la-

ORCID: 0000-0003-2872-3908

bels include mutation free energies from alchemical free-energy perturbation (FEP), Rosetta mutation scores, ThermoMPNN stability scores, Rosetta scores on variants proposed by ESM2 or sampled at random, and an MD-derived Q-value that summarizes native-contact persistence. To keep the comparison target-centered, every candidate setting is selected on experimental Tm validation data, and every final claim is evaluated on the same held-out experimental Tm test set with paired error statistics.

The results are label-dependent. FEP mutation free energies, although relative and mutation-level, give the clearest improvement in absolute Tm prediction, whereas an MD-derived Q-value does not improve under the same protocol. Rosetta-scored ESM2 proposals provide only a weaker exploratory signal. These results extend the Sim2Real argument to a protein-engineering setting in which the computational and experimental labels differ in physical meaning. In this setting, computed data are useful only when they encode information that transfers to the experimental property of interest.

Materials and Methods

Experimental and technical design

The study tests whether a computational label can improve experimental nanobody Tm prediction when that label is physically related to Tm but not identical to it. We used a multi-task transfer-learning design in which experimental Tm prediction was the target task, and each computational label was a computational task that shared part of the network with the target. Throughout, model selection and every reported number were tied to experimental Tm performance. Computational-task losses shaped the shared representation during training, but computational-task validation errors were never used to pick checkpoints or hyperparameters.

Experimental Tm data set and split definitions

Experimental Tm labels came from the public nanobody benchmark NbBench, which provides a thermo-seq task with predefined splits (25). We deliberately assigned the smallest partition to training and the largest to the held-out test set, giving 57 training, 114 validation, and 396 test sequences. This places the model in the low-data regime that the study targets. Scarce experimental Tm labels, not abundant ones, are the condition under which simulation-derived supervision is expected to help. The large test set is a direct benefit of this choice, because it makes paired error comparisons and label-count trends easier to resolve.

Split definitions. The training split fit model parameters. The validation split drove checkpoint selection, early stopping, and hyperparameter choice. The test split stayed untouched until the settings were fixed and was used only for final reporting. The same rule held for Tm-only models and for multi-task transfer models.

Target-label scaling. The target CSV files held an amino-acid sequence column and a Tm label in degrees Celsius. During training, Tm labels were mapped to a scaled regression target by robust scaling followed by linear rescaling to [0,1], with the scaler fit only on the 57 training labels. The fitted scaler was then applied to validation labels during training. At evaluation, predictions were inverse-transformed back to degrees Celsius before errors were computed, so the validation and test label distributions never entered the scaling transform.

Experimental-label scaling reference. For this reference, the Tm training split was subsampled to the requested number of labels using the run seed, while the validation and test splits were left intact. These runs give a target-label baseline for judging whether computational-label scaling moves in the expected direction.

Computational-label data sets

Computational labels were loaded as computational tasks, not as additional Tm measurements.

Computational-label classes. The computational labels were mutation free-energy changes from alchemical free-energy perturbation (FEP), structure-based mutation-effect scores from Rosetta, learned mutation-effect scores from ThermoMPNN, Rosetta scores on ESM2-proposed and random variants, and an MD-derived Q-value summarizing native-contact persistence. All computational labels were preprocessed to [0,1]-scaled regression targets and entered the model only through their own computational-task heads.

Processed computational-table sizes. Mutation-effect computational labels were organized as two template-specific tables derived from the 1MEL and 4IDL nanobody structures. FEP, Rosetta, and ThermoMPNN each contained 435 and 409 processed rows for the two templates. ESM2-proposed variants and random variants scored by Rosetta each contained 1000 and 1000 processed rows. The MD-derived Q-value computational label came from a separate panel of simulated single-domain antibody structures drawn from SAbDab, the Structural Antibody Database (27); its processed table contained 1143 analysis-ready rows. Row counts exclude headers.

Sequence-overlap check. We checked exact sequence overlap between the computational-label tables and the NbBench splits. The mutation-effect tables and the Rosetta-scored variant tables, which carry explicit sequence columns, had no exact match to any NbBench training, validation, or test sequence. The SAbDab-derived MD table contained a few exact matches, including eight test-set sequences. Because the MD Q-value computational label did not improve final Tm prediction, this overlap cannot account for the positive FEP result; we report it for transparency.

Computational-label sampling. For a mutation-effect experiment with requested count n, up to n rows were sampled independently from each of the two template tables using the run seed, and the two tables were given separate task identifiers. For an MD-derived Q-value experiment with count n, n rows were sampled from the single Q-value table. Sampled rows were then split 80/20 into computational-training and computational-validation rows with the same seed. The

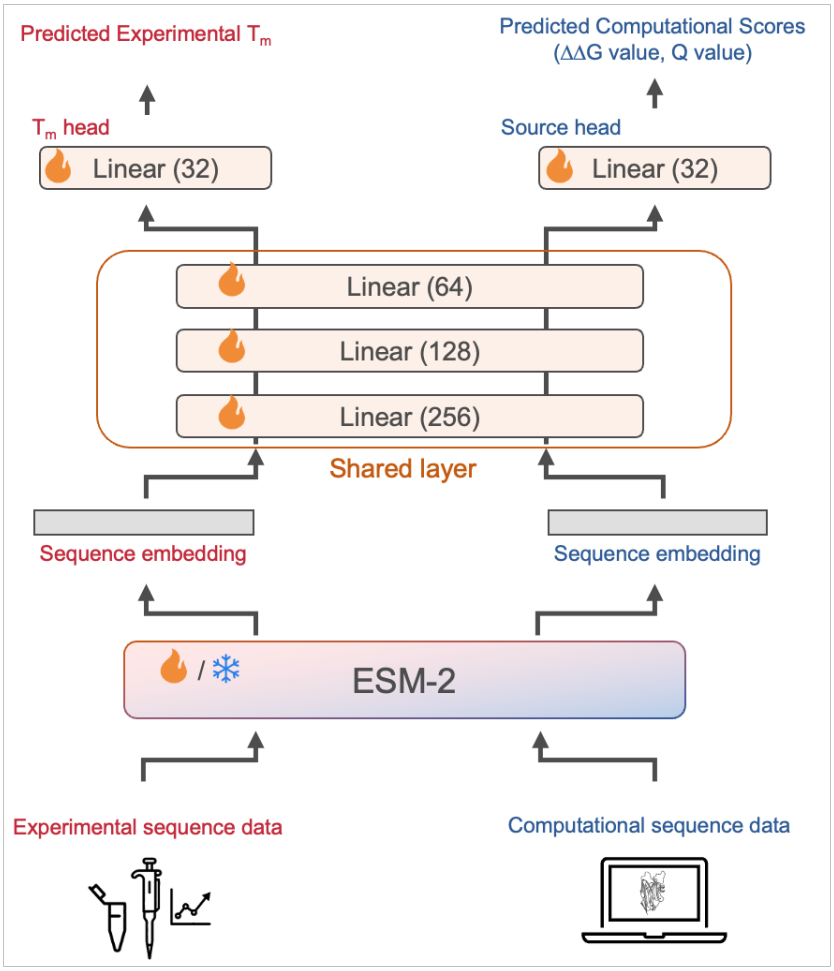

Fig. 1. Multi-task transfer-learning architecture for using simulation data in experimental Tm prediction. Experimental sequence data (left) and computational sequence data (right) are encoded by a shared ESM2 protein language model into per-sequence embeddings. The embeddings pass through a shared multilayer perceptron and then split into two task-specific heads: a Tm head that predicts the experimental melting temperature, and a computational head that predicts the computational scores (mutation $\Delta\Delta G$ values or an MD-derived Q-value). Only the experimental Tm head defines the final objective, while the computational head lets the simulation-derived labels reshape the shared representation during training. Flame and snowflake icons mark trainable and frozen parameters, respectively; the ESM2 encoder is either fine-tuned (hot) or kept frozen depending on the configuration.

computational-validation rows were kept for monitoring and control analyses, but in the reported protocol the validation data passed to the trainer for checkpoint selection and early stopping were filtered down to the experimental Tm validation rows.

SAbDab-derived MD simulations and Q-value labels

The MD computational labels were built from nanobody structures selected from a SAbDab-derived summary table (27). We took one heavy-chain nanobody structure per PDB entry when an IMGT-renumbered coordinate file was available and the chosen chain could be verified in that file. The 400 K simulation panel held 1146 nanobody systems. Requiring a usable trajectory, topology, reference structure, a sequence of at least 50 residues, and at least one selected native contact left 1143 trajectories in the processed table.

Each system was prepared from the selected nanobody chain alone. We used the IMGT-renumbered PDB as the input structure, extracted the heavy chain, dropped non-protein components, left the termini uncapped, and assigned protonation at pH 7.4. Structure cleaning and missing-atom handling used PDBFixer, followed by pdb2pqr/PROPKA for pH-aware protonation and Amber-compatible atom naming when available (28, 29). Systems were solvated in explicit OPC water with 15 Å padding and 0.15 M NaCl, and Amber topologies were built with the ff19SB protein force field and OPC water model (30–32).

Trajectories were run with OpenMM on CUDA devices (33). Each system was equilibrated at 300 K with 1 ns NVT and 1 ns NPT at 1 atm, using a 2-fs timestep without hydrogen-mass repartitioning. The 400 K branch was then started from the 300 K equilibrated state through two restrained NVT relaxation stages. The first was 1 ns at 400 K with Cα restraints, followed by a second 1 ns 400 K restrained NVT. Production ran 100 ns of restraint-free 400 K NVT with hydrogen-mass repartitioning (4 amu hydrogen mass), a 4-fs timestep, Langevin middle integration at a 1 ps^{-1} collision rate, PME electrostatics with a 1.0-nm nonbonded cutoff, and coordinate/energy output every 100 ps. The MD computational labels use this 400 K production branch.

Q-values were extracted from the 400 K production trajectories with MDTraj (34). For each trajectory, the topology was loaded from the Amber parameter file and the native reference was frame 0 of production. A smooth native-contact fraction (35) was computed from backbone-heavy-atom contacts and averaged over the terminal portion of the trajectory, then scaled to [0,1] before training. The exact contact selection, contact function, averaging window, and column convention are given in Supplementary Methods.

FEP mutation free-energy labels

The FEP computational labels were mutation free-energy changes for variants of the 1MEL and 4IDL nanobody templates, computed with the same protocol as our related nanobody thermostability study. In brief, alchemical mutation simulations were run with NAMD 3 using GPU acceleration (36). Input systems were prepared in VMD (37) with a customized AlaScan workflow (38), and coordinates were initialized from AlphaFold 3 models of the wild-type sequences (39). Systems used the CHARMM36 protein force field (40), TIP3P water (41), and 150 mM NaCl.

Each transformation used a dual-topology mutation setup with a CHARMM36 hybrid-residue topology, split into 20 equally spaced λ windows from wild type to mutant. Forward and backward transformations were run for 1 ns per direction with a 2-fs timestep and 200 ps of per-window equilibration before data collection, all in the NPT ensemble at 300 K and 1 atm with Langevin temperature control and a Langevin piston barostat. Bonds to hydrogen were constrained with SHAKE (42), and long-range electrostatics used particle mesh Ewald (43). Relative free energies were estimated with the Bennett acceptance ratio (44). The processed tables retain the raw mutation free-energy value, its negated value, and the network-facing [0,1]-scaled label; under this convention, larger processed labels correspond to more favorable mutation free energies.

Rosetta and ThermoMPNN mutation-effect labels

Rosetta mutation-effect labels were generated with the Rosetta `ddg_monomer` application (45). For each template, the PDB structure was cleaned to the target chain, variant sequences were compared with the wild-type sequence, and the differences were written to Rosetta mutation-list files. The calculation used full-atom Rosetta with the `ref2015` score function (46), 50 iterations, Cartesian minimization enabled, local-only optimization disabled, side-chain-minimization-only disabled, minimum-score aggregation, and no PDB dumping. The same protocol scored the template mutation-effect tables, the ESM2-proposed variants, and the random-variant control.

For the ESM2-proposed computational label, ESM2 650M sampled 100,000 two-mutation variants from each wild-type template, ranked them by pseudo-log-likelihood, and kept the top 1% for Rosetta scoring (14). The random-variant control used roughly 1000 two-mutation variants with positions and substitutions chosen at random, scored by the same Rosetta workflow.

ThermoMPNN computational labels were computed from the same template-specific variant sets with ThermoMPNN, a stability-change predictor trained by transfer learning from larger protein-stability data (47), and converted to the same processed CSV format as the other mutation-effect computational labels.

Neural-network architecture

All models used an ESM2 protein language-model encoder (14) with lightweight regression heads. Unless stated otherwise, the encoder was the 8M-parameter ESM2 model. Sequences were tokenized with truncation and padding to 160 residues. The first-token encoder representation fed a shared multilayer perceptron with hidden dimensions 256, 128, and 32, ReLU activations, and dropout. A scalar Tm head predicted the scaled target label; in computational-label transfer models, additional scalar heads predicted the computational labels from the same shared representation.

The Tm-only baseline used the encoder, the shared trunk, and the Tm head with experimental Tm labels alone. Computational-label transfer models kept this target path and added computational heads. The main FEP model used separate computational heads for the two template-specific mutation-effect tasks, the best design in our computational-head comparison. As controls we also tried a shared mutation-effect head, a shared head conditioned on a template embedding, and a shared head with template-specific affine calibration, along with residual and latent computational-fusion variants; the main result used the original shared-trunk architecture.

Hot-encoder and frozen-encoder training

We evaluated two encoder regimes. In the hot-encoder regime, the ESM2 encoder and the regression heads were optimized jointly, with the encoder given a smaller learning rate than the freshly initialized trunk and heads so that the pretrained representation was updated more gently. In the final FEP setting, the selected encoder learning rate was 3×10^{-5} and the head learning rate was the default 3×10^{-4}.

In the frozen-encoder control, all ESM2 parameters were fixed. The encoder was run once to precompute sequence embeddings for the target and computational examples, and training then optimized only the trunk and task heads. This control asks whether the computational labels still help when the pretrained representation is used as a fixed feature extractor. Encoder-size controls repeated selected Tm-only and FEP settings with 35M- and 650M-parameter ESM2 encoders.

Loss function and multi-task objective

Each training example consisted of a sequence x_i, scaled label y_i, and task identifier t_i. The target Tm task had one identifier, and each computational task had its own. A minibatch could mix target and computational examples, but every example contributed loss only through the head for its task.

The per-task regression loss was Huber loss with $\delta = 1.0$ on scaled labels. The default multi-task objective used learn-

able task-uncertainty weights (48). For task k with loss L_k and learned log-scale s_k, the contribution was

$$\frac{L_k}{2\exp(2s_k)} + s_k,$$

and the total loss summed over the tasks present in the mini-batch. Fixed computational-task weights were also evaluated as controls, particularly for the MD-derived labels.

Training, early stopping, and hyperparameter selection

Models were trained with AdamW, cosine learning-rate scheduling, batch size 16, weight decay 0.04, 100 warmup steps, and at most 400 epochs. The default head learning rate was 3×10^{-4}, the default hot-encoder learning rate was 1×10^{-4}, and the default dropout was 0.15. Training used mixed precision on CUDA devices, with checkpoints saved and evaluated once per epoch.

Within each run, the best checkpoint was the one with the lowest mean squared error on the experimental Tm validation examples, and early stopping used the same metric with a patience of 15 epochs. Computational-label rows were present in the training set, but they were removed from the validation data passed to the trainer for checkpoint selection and early stopping. This is the implementation-level step that keeps reported checkpoints selected by target-task validation performance rather than by computational-task performance.

Hyperparameters were likewise selected on the experimental Tm validation split. Candidate settings varied the encoder learning rate, head learning rate, dropout, loss-weighting mode, computational-head design, and selected architecture controls. The search covered encoder learning rates of 3×10^{-5}, 1×10^{-4}, and 3×10^{-4}; a lower head learning rate of 1×10^{-4} paired with encoder learning rate 3×10^{-5}; dropout rates of 0.05, 0.15, and 0.30; and fixed computational-task loss weights where relevant. Each candidate was evaluated as a three-seed ensemble on the Tm validation split, and the selected setting for each computational class was then evaluated on the held-out test split as a ten-seed ensemble.

Independent run seeds controlled model initialization, target-training subsampling in the target-label scaling experiment, computational-label sampling, and computational-task train/validation splitting. Final comparisons used seeds 1 to 10. The label-count curves used fixed computational-specific training recipes while varying the number of computational labels, rather than retuning a separate model at every label-count point.

Final evaluation and statistics

Final evaluation used the same 396 held-out Tm test examples for every condition. For each condition, ten independently seeded models produced scaled Tm predictions for every test sequence; these were averaged in scaled-label space and inverse-transformed to degrees Celsius. The primary metric was mean absolute error (MAE) in degrees Celsius. Root mean squared error, coefficient of determination, and Spearman and Pearson correlations were also computed, but MAE anchored the main comparisons because it reads directly as an average temperature error.

Confidence intervals came from nonparametric bootstrap resampling over held-out test proteins. For a single condition, the absolute-error vector from the ten-seed ensemble was resampled with replacement 10,000 times using a fixed random seed, and the reported 90% interval is the 5th to 95th percentile range of the bootstrapped MAE values. We report 90% intervals because the comparisons are directional, paired, and intended to emphasize effect sizes rather than binary significance thresholds; the bootstrap summaries retained by the analysis code allow other interval levels to be recomputed from the same resamples.

For paired comparisons, the same bootstrap indices were applied to the absolute-error vectors of the candidate and reference conditions, and the candidate-minus-reference MAE difference was computed for each bootstrap sample. This paired bootstrap preserves the matched-test-example structure of the comparison. Throughout, ΔMAE is candidate minus reference, so negative values mean lower error than the reference. The evaluation code also reports the one-sided tail probability, defined as the fraction of paired bootstrap samples with ΔMAE above zero, but we emphasize effect sizes and 90% confidence intervals.

Implementation details

The pipeline builds the target and computational tasks, applies target-label scaling, performs seeded computational-label sampling, filters validation data for target-task checkpoint selection, evaluates seed ensembles, and writes bootstrap statistics. Each run emits a machine-readable summary with the command-line arguments, resolved model settings, per-point metrics, bootstrap intervals, and paired comparisons. These summaries feed the figure-generation workflow directly and will be archived with the processed data for reproducibility.

Results

A target-centered framework for using simulation labels in Tm prediction

We first built the evaluation framework. The target task was nanobody Tm regression from amino-acid sequence, and computational labels were treated as computational prediction tasks, not as stand-ins for experimental Tm. Each model used the pretrained protein sequence model ESM2 to turn a sequence into a learned representation, which a Tm head and a computational head shared. The computational task could reshape the representation while the final objective stayed anchored to experimental Tm (Fig. 1).

We used the benchmark split as a deliberately low-data target setting, with 57 experimental Tm labels for training, 114 for validation, and 396 for held-out testing. Every candidate setting was selected on the experimental Tm validation set, and final numbers came only from the held-out Tm test set. Improvements were measured as paired differences in absolute error on the same test examples. This target-

centered protocol guards against a trap specific to simulation-to-experiment learning. A computational task can be learnable and physically sensible yet do nothing for the experimental phenotype.

Transfer depends on label content, not data volume

We next varied the number of experimental and computational labels to see whether more computational data meant better Tm prediction. Adding experimental Tm labels helped the Tm-only model, as expected (Fig. 2). The target-label sweep used 10, 20, 30, 40, and 57 experimental Tm training labels; the FEP sweep used 10, 40, 80, 160, and 320 rows per template-specific computational table; and the MD Q-value sweep used 10, 40, 80, 160, 320, and 640 rows from the single MD computational-label table. FEP mutation free-energy labels showed a beneficial trend. The largest FEP setting gave the best held-out Tm error among the FEP points. The curve was not strictly monotonic, so it does not establish a clean scaling law. The supported conclusion is narrower. In this low-data Tm setting, the largest tested FEP computational-label pool transferred best when models were selected and judged through the experimental Tm endpoint.

The MD-derived Q-value behaved differently. This label measures how well native contacts persist during high-temperature MD trajectories. Adding more of these labels did not reproduce the FEP trend, even though they too came from atomistic simulation. The best point within the MD Q-value sweep was close to the Tm-only reference, whereas the best FEP point was lower. Thus, label count alone was not enough; the computational label had to encode information that transferred to the experimental endpoint.

Two controls further separated computational-label content from model capacity. First, increasing the ESM2 encoder from 8M to 35M or 650M parameters did not improve held-out Tm accuracy in the Tm-only setting and did not reproduce the 8M FEP result in the FEP setting. Second, comparing frozen and hot encoders showed that FEP improved over the corresponding Tm-only baseline in both regimes, with the larger gain when the encoder was updated. The MD Q-value did not show the same consistent behavior. These overview results motivated the all-atom comparison in Fig. 3 and the lower-cost computational-label comparison in Fig. 4.

All-atom computational labels depend on the simulated quantity

The overview comparison showed a clear split between two all-atom computational labels: FEP mutation free energies improved Tm prediction, whereas the simple MD-derived Q-value did not. We therefore asked whether the MD result reflected the trajectories themselves or the scalar label extracted from them. Fig. 3a places the Tm-only and FEP reference points on the same held-out test scale as each MD-derived descriptor, now evaluated at both 300 K and 400 K wherever trajectory data exist. The Tm-only reference reached 6.62°C MAE, and the FEP reference reached 6.26°C MAE. No MD descriptor matched FEP. The lowest descriptor error came from 300 K disulfide-distance fluctuation (6.48°C MAE), followed by the sequence-only CDR3-length control (6.52°C MAE). Temperature mattered in a descriptor-specific way: the Q-value transferred slightly better at 400 K than at 300 K (6.67 versus 6.72°C MAE), consistent with its wider dynamic range at the higher temperature, whereas the structural-fluctuation descriptors were simulated at 300 K, where native-state motions are defined (disulfide-distance 6.48°C, RMSF max 6.65 versus 6.69°C at 400 K, CDR3-residue fluctuation 6.66°C). Across descriptors these temperature shifts were small relative to the bootstrap intervals.

These results narrow the interpretation of the MD comparison. The negative result for a terminal Q-value should not be read as evidence that trajectory information is useless. Rather, it shows that the transfer-learning model is sensitive to how an MD trajectory is compressed into a computational label. Panel b shows that adding more MD Q-value labels does not systematically lower held-out Tm error at either temperature: the 300 K Q-value is saturated near one (folded ensembles, little usable variation), and even the more variable 400 K Q-value fails to scale toward the FEP regime. The temperature and descriptor choice change the information available to the model, so the design problem is not only to run MD but also to choose which dynamical quantity, at which temperature, should supervise the target predictor. In the present screen, FEP remained the strongest all-atom computational label, but the descriptor controls show that MD-derived labels can move closer to the useful regime when they encode the appropriate structural variation.

Structure-based and proposal labels transfer weakly

We next looked at computational labels outside the FEP and MD pipelines. Rosetta scores on ESM2-proposed variants reached 6.45°C MAE (paired change, -0.17°C; 90% confidence interval, -0.31 to -0.03°C), and ThermoMPNN stability scores reached 6.46°C MAE (paired change, -0.15°C; 90% confidence interval, -0.30 to -0.01°C; Fig. 4). Rosetta mutation scores reached 6.53°C MAE (paired change, -0.09°C; 90% confidence interval, -0.20 to +0.02°C), and random variants scored by Rosetta reached 6.51°C MAE (paired change, -0.10°C; 90% confidence interval, -0.22 to +0.03°C). All were weaker than FEP, and the two Rosetta controls had confidence intervals that crossed zero. Label-count sweeps for these sources (Fig. 4b) showed at most a shallow, noisy decrease in held-out error as more computational labels were added, none reproducing the cleaner FEP scaling trend.

Across the label-count sweep, the ESM2-proposed variants scored by Rosetta edged out the random variants, with a paired MAE difference of -0.07°C in favor of the ESM2-proposed set. The confidence interval crossed zero, so we read this as exploratory rather than as evidence that the present generator improves variant selection. It does show, though, that language-model proposals can be annotated by physics-based scoring and used as computational labels for the Tm predictor. We return to this feasibility point in the Discussion.

Across the labels we tested, the strongest and most con-

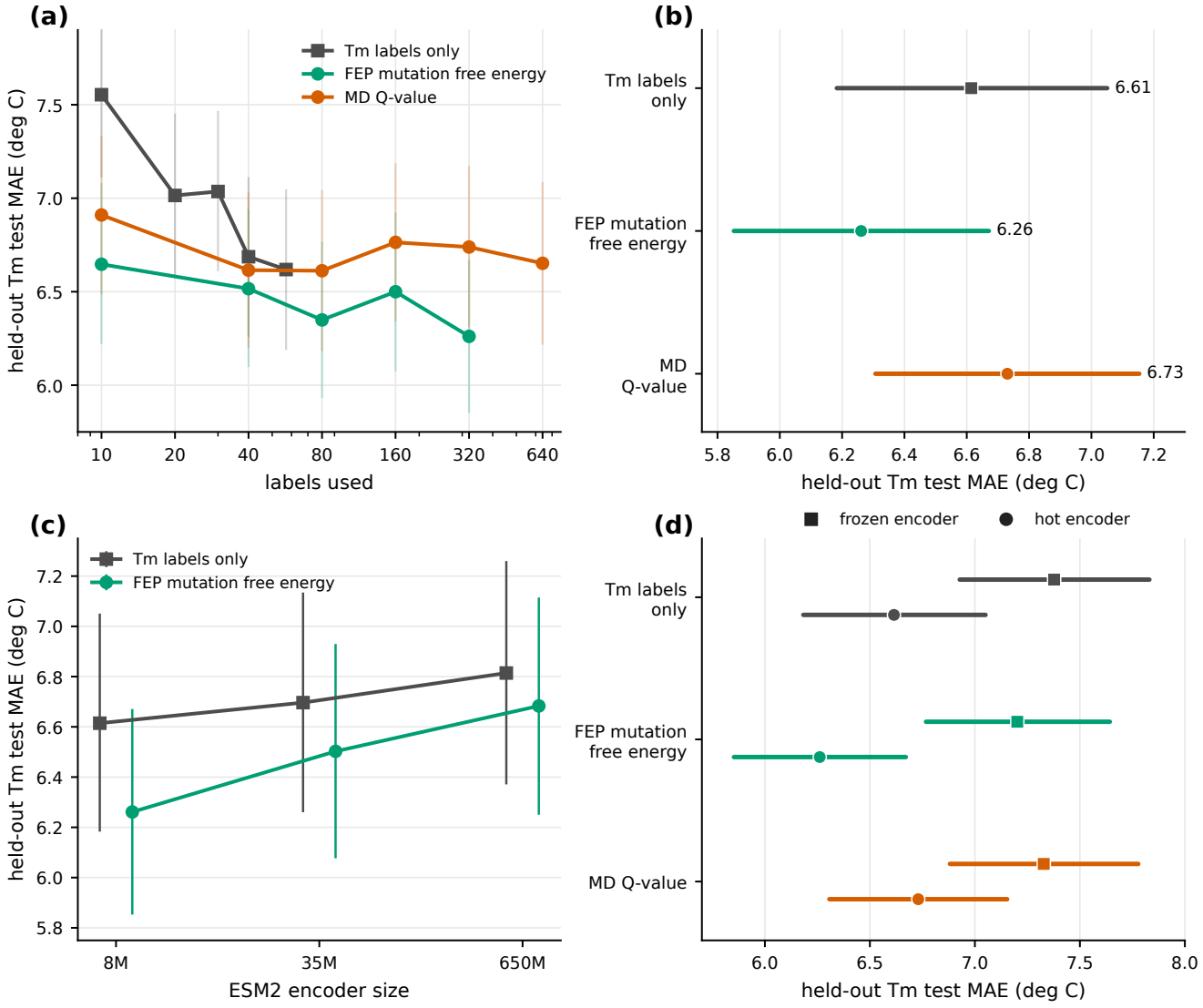

Fig. 2. Overview of computational-label transfer in the low-data Tm setting. (a) Label-count sweeps for experimental Tm labels, FEP mutation free-energy labels, and MD Q-value labels. (b) Held-out test MAE of the final validation-selected model for each condition. (c) ESM2 encoder-size controls for Tm-only and FEP-assisted training. (d) Held-out test MAE for Tm-only, FEP, and MD Q-value labels under frozen and hot encoders. In all panels, points show ensemble MAE and intervals show 90% bootstrap intervals.

sistent improvement came from FEP mutation free energies. Other structure-based labels and proposal-set scores gave weaker gains, and the MD-derived Q-value gave none. The pattern is selective, not additive. A computational label helps only when the information it carries transfers to the experimental Tm endpoint.

Discussion

This study asked how computed stability labels should be used when the experimental target is scarce and the computed quantity is related to, but not the same as, that target. In low-data nanobody Tm prediction, FEP mutation free-energy labels gave a modest but consistent paired improvement in held-out Tm error. The same default target-centered protocol did not benefit from a raw MD-derived Q-value, and larger ESM2 encoders did not reproduce the FEP gain. This pattern points to selective transfer. Computational labels must carry stability information that transfers to the experimental endpoint under validation rules defined by that endpoint. The absolute gain was small in degrees Celsius, so it should be read as a statistically consistent improvement in a low-data predictor, not as evidence that individual nanobody designs would show an assay-level stability gain of the same magnitude.

These results connect nanobody Tm prediction with Sim2Real learning. Existing nanobody Tm predictors rely mainly on PLM fine-tuning or structure-based features (25). Materials-informatics Sim2Real studies show that large computed databases can improve small experimental target sets when the computational and experimental domains are related (3). Our setting adds a stricter test. The computational label and the target label differ in physical meaning. FEP provides relative mutation free energies, whereas the target is an absolute melting temperature. The study shows that such

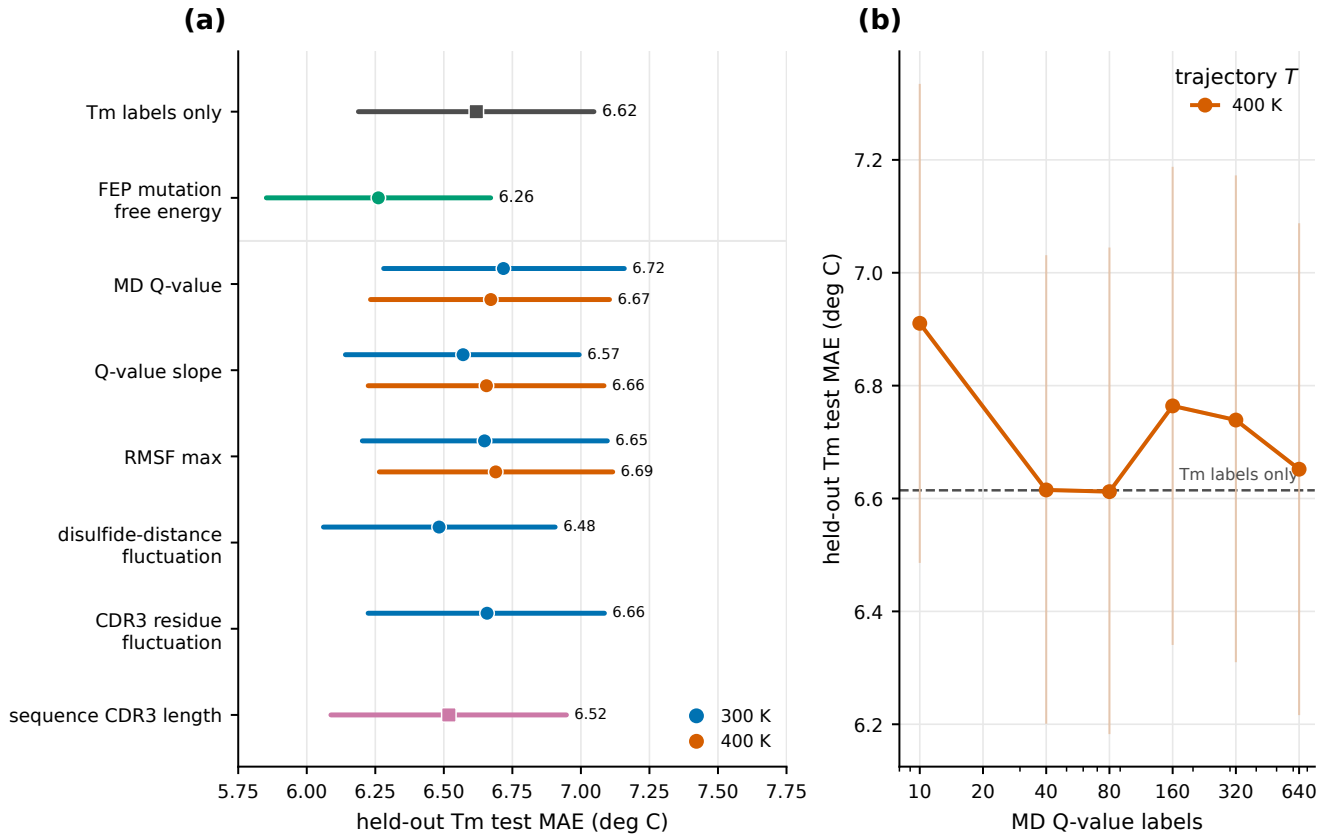

Fig. 3. MD descriptor choice controls transfer to Tm prediction. (a) Held-out test MAE for each MD-derived descriptor used as the sole auxiliary task, evaluated at 300 K (blue) and 400 K (orange) wherever trajectory data exist; disulfide-distance and CDR3-residue fluctuations were simulated only at 300 K, and CDR3 length is sequence-derived. Tm-only and FEP conditions are shown as references. All descriptors share the same residual multi-task setup, so this panel is a like-for-like descriptor comparison; its absolute values come from a different training series than the source-screen models in Fig. 2 and Fig. 4 and are not meant for cross-figure numerical comparison. Points and intervals show ensemble MAE and 90% bootstrap intervals. (b) Held-out test MAE versus the number of MD Q-value labels for trajectories at 300 K and 400 K, with Tm-only as a dashed reference (hot encoder; the 400 K curve is the sweep also shown in Fig. 2). Adding more Q-value labels does not systematically reduce Tm error at either temperature. Points and intervals show ensemble MAE and 90% bootstrap intervals.

a computational label can augment scarce Tm supervision, whereas other available computed labels transfer weakly or not at all.

The helpful label measured a different quantity from Tm. FEP estimates how a mutation changes free energy, producing a relative $\Delta\Delta G$ value for that mutation. Tm is instead the absolute temperature at which the folded and unfolded states balance under a given assay. The two are not interchangeable. They do, however, read out related aspects of the same stability landscape. Sparse Tm measurements anchor the absolute scale, while mutation free energies provide local directions in sequence space. In this study, that shared physical content was sufficient for $\Delta\Delta G$ supervision, selected only by Tm validation performance, to improve held-out Tm prediction even though the computational labels came from template variants different from the Tm test sequences.

This interpretation also separates computational-label content from generic auxiliary-task regularization. A raw MD-derived Q-value supplied an additional simulation task but did not improve the target endpoint under the default protocol. The gain tracks the specific stability-change information in $\Delta\Delta G$.

Three controls sharpen this reading. First, the FEP label-count sweep was best at the largest tested setting, but it was not smooth enough to claim a biomolecular scaling law; the firmer conclusion is label dependence. This both echoes and complicates the materials-informatics case, where transferred error can scale with the size of a related computational database (3). In our protein setting, the useful labels were mutation free energies, not extra Tm measurements. Second, model size was not the lever. The 35M and 650M ESM2 encoders did not beat the 8M encoder in either the Tm-only or FEP settings, and the best model paired the 8M encoder with FEP labels, consistent with the broader observation that fine-tuning rather than raw capacity can drive PLM-based protein-property prediction (26). Third, the frozen-encoder control locates the signal. FEP labels helped even with a fixed representation, but the best FEP model required encoder adaptation. The labels work best when they are allowed to reshape the representation toward stability-relevant directions while still being selected on experimental Tm.

The MD result calls for the same specificity. Other MD-derived descriptors may behave differently, but a raw high-temperature native-contact Q-value did not transfer under the default target-centered protocol. By compressing each trajectory to a single terminal contact-persistence statistic, it may

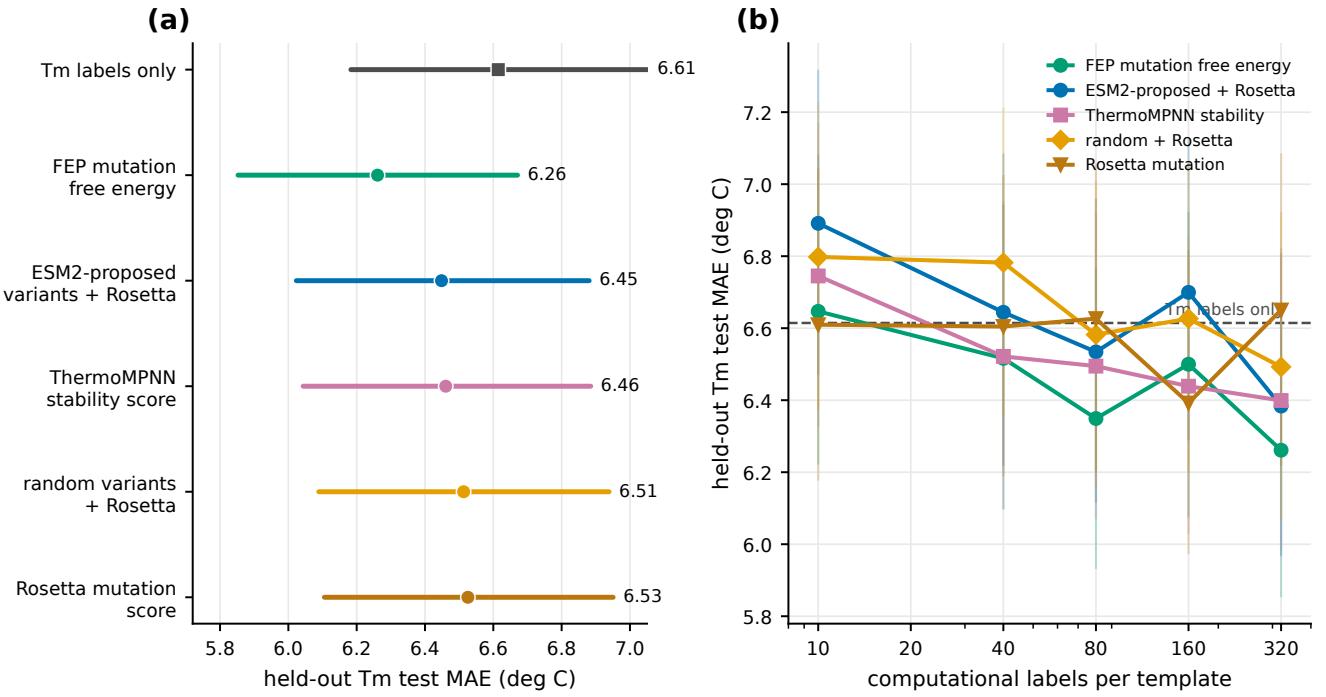

Fig. 4. Structure-based labels and language-model-proposed variants. (a) Final held-out test MAE (validation-selected models) for Tm-only learning, FEP, and the structure-based and variant computational labels. (b) Held-out test MAE versus the number of computational labels per template for each mutation-effect and variant source, with Tm-only shown as a dashed reference. These are fixed-configuration label-count sweeps (the FEP curve is the 10-seed sweep of Fig. 2), so their largest-label points need not coincide exactly with the validation-selected models in (a). Points and intervals show ensemble MAE and 90% bootstrap intervals.

miss mutation-specific energetics that govern how sequence changes shift stability. The descriptor controls support this point. Q-value slope was slightly better than the raw Q-value, and disulfide-distance fluctuation was better still, although neither reached the FEP reference. A simulation campaign aimed at target transfer should choose trajectory observables for their expected physical relevance to the endpoint, rather than for label count alone.

The Rosetta-scored proposal-set result is best read as a feasibility test for design workflows. Rosetta scores on ESM2-proposed variants improved Tm prediction over Tm-only learning, but the direct comparison with random variants stayed within noise, so we do not claim that the present generator has already optimized nanobody thermostability. The supported conclusion is that language-model proposals can be annotated by a physics-based score and used as supervised computational labels for a Tm predictor.

That supervised-scoring step is the only part of a design loop we tested. A full loop would let a generator propose variants, physics-based calculations annotate them, the Tm predictor update its estimate of experimental stability, and that estimate drive the next round of proposals, connecting this transfer-learning result to active or reinforcement learning. Those closed-loop steps, and experimental validation of newly proposed high-Tm nanobodies, are beyond the present scope.

The practical implication is not the size of the improvement, which was modest, but the criterion it suggests for data collection. Under target-centered evaluation, a computational label helped only when its physical quantity was close to the target ($\Delta\Delta G$), not when it was simulation-derived (the MD Q-value) or backed by a larger model alone. A data campaign aimed at a scarce phenotype such as Tm should be chosen for physical relatedness to that phenotype, and then validated by held-out target performance, before committing to scale.

For nanobody Tm specifically, three priorities follow. First, relative stability at scale for VHH-length scaffolds. The strongest computational label here, $\Delta\Delta G$, is the quantity that high-throughput proteolysis can already measure for on the order of 10^5 domains (22), but those domains are short (40 to 72 residues) whereas VHH domains are longer; extending such assays, or computational $\Delta\Delta G$, to VHH-length scaffolds would supply the large, physically matched computational-label pool that our two-template FEP set only begins to sample. Second, template and mutation diversity. Our FEP labels came from two templates, enough to show transfer but not to scale it; $\Delta\Delta G$ spanning many templates and mutations, broad enough to cover the sequence neighborhood of the Tm target, is the natural next dataset. Third, modest but well-distributed Tm anchors. The target side here was 57 sequences, and a few hundred Tm measurements spread across the relevant stability range would fix the absolute scale that $\Delta\Delta G$, being relative, cannot set on its own.

These choices also trade off against cost, although this study did not measure cost systematically. Learned predictors and structure-based scores can label large variant sets much faster than all-atom alchemical calculations, and they gave modest transfer signals in our experiments. Such labels may be useful for broad, inexpensive computational-label pools, but the main risk is saturation. A proxy can carry the model

only as far as its own accuracy and physical relevance allow, so adding more cheap labels may stop helping once that ceiling is reached. All-atom FEP sits at the other end of this tradeoff. It is more expensive per mutation, but it gave the strongest per-label transfer observed here. This points to a tiered strategy in which learned or structure-based labels provide breadth, while all-atom calculations can be reserved for regions where a more directly stability-related label is worth the extra cost.

The negative results also guide follow-up work. Collecting more coarse trajectory summaries, such as a single native-contact Q-value, is unlikely to repay the simulation cost. If molecular dynamics is used as a computational label, the observable should be designed to report mutation-specific energetics rather than a global stability proxy. Enlarging the language model is likewise not a substitute for physically matched labels.

A supplementary control is consistent with this selectivity. Combining FEP and MD Q-value labels in the same model did not improve over FEP alone (Supplementary Fig. 3d), reinforcing that computational-task selection should prioritize quality over quantity. Conversely, when the MD Q-value model was tuned separately at each label count on the experimental validation split, its performance improved and crossed the Tm-only baseline at the largest setting (Supplementary Fig. 4b). This suggests that the failure of MD labels under our default protocol is partly methodological. A fixed hyperparameter recipe disadvantaged the MD computational label rather than proving that the computational label is purely uninformative. The FEP result was retained under the same fixed recipe and the validation-tuned FEP model matched it.

These costs should ultimately be weighed against experimental value. Minami et al. formalized this idea as an equivalent sample size, defined as the number of simulation labels that substitute for one experimental label along an equal-error contour (3). Our data permit only a coarse version of that calculation. Near the operating point of this study, 57 Tm labels and 320 FEP labels per template, the empirical target-label curve gives an approximate local slope of 0.009°C per Tm label. Matching that slope with FEP labels would require on the order of 30 labels per template, or roughly 60 labels in total. The MD-derived Q-value curve does not decrease with added labels under the default protocol, so the same calculation gives little experimental-label equivalence for that computational label. Because the FEP curve is non-monotonic and the computational costs were not measured systematically, these values should be treated as planning-scale estimates rather than a cost-benefit analysis.

A large and diverse $\Delta\Delta G$ dataset would also let the central quantitative question be tested directly. Whether held-out Tm error falls predictably as the computational-label pool grows, the scaling behavior that makes simulation databases worth building in materials informatics (3), could not be settled here because our pool was small and the label-count curve was not monotonic. Establishing that relationship would turn the qualitative observation that more physically relevant data helps into a quantitative guide for how much such data to collect.

Limitations

Several limitations bound the interpretation. The experimental Tm training set is small, so the conclusions should be retested on other nanobody collections and protein families. The computational-label comparison covers only the labels available here; other free-energy protocols, stability predictors, or trajectory descriptors may transfer differently. The FEP curve supports a beneficial trend at the largest setting but does not establish a clean scaling exponent. The MD computational table contained a small number of exact NbBench sequence overlaps, and the negative MD result should be rechecked after removing those rows in a larger follow-up analysis. No new experimental measurements were made to validate high-stability candidates. Because raw trajectories may be too large to deposit in full, reproducibility will rely on processed data, scripts, and a clearly reported simulation protocol.

For low-data experimental protein-property prediction, simulation labels are best treated as design choices rather than generic data augmentation. Their value depends on whether they encode information that improves the target task under target-centered validation. Here, mutation free energies did that for nanobody Tm. The resulting strategy is to use experimental data to define the target phenotype, use simulation to annotate physically meaningful perturbations around it, and judge the combined model only on held-out experimental data. This strategy can support future simulation-assisted and closed-loop protein-engineering workflows.

Conclusions

We have shown that simulation-to-real transfer learning improves low-data nanobody Tm prediction when the computational label is physically matched to the target, and that the benefit is selective rather than generic. FEP mutation free energies gave a consistent 0.35°C improvement in held-out Tm MAE, while an MD-derived Q-value did not transfer under the same protocol, and enlarging the ESM2 encoder did not reproduce the FEP gain. These results support a conservative criterion for allocating simulation effort in protein engineering. The most defensible computational labels are those whose physical content is related to the experimental endpoint, whose value is validated under target-centered model selection, and whose performance improves when informative labels are scaled.

ACKNOWLEDGEMENTS

This work was supported by JSPS KAKENHI Grant 23H03412 and JST NBDC Grant JPMJND2603, and partly supported by MEXT as "Program for Promoting Researches on the Supercomputer Fugaku" (Development and application of large-scale simulation-based inferences for biomolecules JPMXP1020230119). Computational resources were provided by the supercomputer Fugaku at the RIKEN Center for Computational Science under Project IDs hp230209, hp240215, and hp250233.

AUTHOR CONTRIBUTIONS

Taihei Murakami and Kentaro Sasaki contributed equally to this work. Yasuhiro Matsunaga conceived and supervised the study, acquired funding, developed the analysis and training code, performed the analyses, interpreted the results, prepared the figures, and wrote the manuscript. Kentaro Sasaki performed computational calculations. Taihei Murakami contributed to draft writing. All authors reviewed and approved the final manuscript.

Declaration of competing interest

Taihei Murakami is an employee of Epsilon Molecular Engineering, Inc. Yasuhiro Matsunaga is a scientific advisor of Epsilon Molecular Engineering, Inc.

Data availability

Processed data required to reproduce the main and supplementary figures are available on GitHub (see Code availability) and will be deposited in Zenodo upon acceptance. Raw simulation trajectories are large and will be made available upon request.

Code availability

Analysis, training, and figure-generation code is available on GitHub at https://github.com/ymatsunaga/sim2real-nanobody-tm (archival release will be deposited in Zenodo upon acceptance).

AI-use disclosure

AI-assisted tools were used to support language editing, code development, and figure preparation. The authors reviewed and take responsibility for all manuscript content, analyses, and conclusions.

References

1. Hironao Yamada, Chang Liu, Stephen Wu, Yukinori Koyama, Shenghong Ju, Junichiro Shiomi, Junko Morikawa, and Ryo Yoshida. Predicting materials properties with little data using shotgun transfer learning. *ACS Central Science*, 5(10):1717–1730, 2019. doi: 10.1021/acscentsci.9b00804.
2. Yuta Aoki, Stephen Wu, Teruki Tsurimoto, Yoshihiro Hayashi, Shunya Minami, Tadamichi Okubo, Kazuya Shiratori, and Ryo Yoshida. Multitask machine learning to predict polymer–solvent miscibility using Flory–Huggins interaction parameters. *Macromolecules*, 56(14): 5446–5456, 2023. doi: 10.1021/acs.macromol.2c02600.
3. Shunya Minami, Yoshihiro Hayashi, Stephen Wu, Kenji Fukumizu, Hiroki Sugisawa, Masashi Ishii, Isao Kuwajima, Kazuya Shiratori, and Ryo Yoshida. Scaling law of Sim2Real transfer learning in expanding computational materials databases for real-world predictions. *npj Computational Materials*, 11:146, 2025. doi: 10.1038/s41524-025-01606-5.
4. Serge Muyldermans. Nanobodies: Natural single-domain antibodies. *Annual Review of Biochemistry*, 82:775–797, 2013. doi: 10.1146/annurev-biochem-063011-092449.
5. Elena Alexander and Kam W. Leong. Discovery of nanobodies: a comprehensive review of their applications and potential over the past five years. *Journal of Nanobiotechnology*, 22 (1):661, 2024. doi: 10.1186/s12951-024-02900-y.
6. Patrick Kunz, Katinka Zinner, Norbert Mücke, Tanja Bartoschik, Serge Muyldermans, and Jörg D Hoheisel. The structural basis of nanobody unfolding reversibility and thermoresistance. *Sci. Rep.*, 8(1):7934, 2018.
7. Tara M Mezzasalma, James K Kranz, Winnie Chan, Geoffrey T Struble, Céline Schalk-Hihi, Ingrid C Deckman, Barry A Springer, and Matthew J Todd. Enhancing recombinant protein quality and yield by protein stability profiling. *J. Biomol. Screen.*, 12(3):418–428, 2007.
8. S Kumar, C J Tsai, and R Nussinov. Factors enhancing protein thermostability. *Protein Eng.*, 13(3):179–191, 2000.
9. Gert-Jan Bekker, Benson Ma, and Narutoshi Kamiya. Thermal stability of single-domain antibodies estimated by molecular dynamics simulations. *Protein Science*, 28(2):429–438, 2019. doi: 10.1002/pro.3546.
10. Ren Higashida and Yasuhiro Matsunaga. Enhanced conformational sampling of nanobody CDR H3 loop by generalized replica-exchange with solute tempering. *Life (Basel)*, 11(12): 1428, 2021.
11. Ammar Mohseni, Maryam Molakarimi, Majid Taghdir, Reza H Sajedi, and Sadegh Hasannia. Exploring single-domain antibody thermostability by molecular dynamics simulation. *J. Biomol. Struct. Dyn.*, 37(14):3686–3696, 2019.
12. Yusuke Tomimoto, Rika Yamazaki, and Hiroki Shirai. Increasing the melting temperature of VHH with the in silico free energy score. *Sci. Rep.*, 13(1):4922, 2023.
13. Alexander Rives, Joshua Meier, Tom Sercu, Siddharth Goyal, Zeming Lin, Jason Liu, Demi Guo, Myle Ott, C. Lawrence Zitnick, Jerry Ma, and Rob Fergus. Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences. *Proceedings of the National Academy of Sciences of the United States of America*, 118(15): e2016239118, 2021. doi: 10.1073/pnas.2016239118.
14. Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, Allan dos Santos Costa, Maryam Fazel-Zarandi, Tom Sercu, Salvatore Candido, and Alexander Rives. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023. doi: 10.1126/science.ade2574.
15. Thomas Hayes, Roshan Rao, Halil Akin, Nicholas J Sofroniew, Deniz Oktay, Zeming Lin, Robert Verkuil, Vincent Q Tran, Jonathan Deaton, Marius Wiggert, Rohil Badkundri, Irhum Shafkat, Jun Gong, Alexander Derry, Raul S Molina, Neil Thomas, Yousuf A Khan, Chetan Mishra, Carolyn Kim, Liam J Bartie, Matthew Nemeth, Patrick D Hsu, Tom Sercu, Salvatore Candido, and Alexander Rives. Simulating 500 million years of evolution with a language model. *Science*, 387(6736):850–858, 2025.
16. Lasse M. Blaabjerg, Maher M. Kassem, Liam L. Good, Nicolas Jonsson, Matteo Cagiada, Kristoffer E. Johansson, Wouter Boomsma, Amelie Stein, and Kresten Lindorff-Larsen. Rapid protein stability prediction using deep learning representations. *eLife*, 12:e82593, 2023. doi: 10.7554/eLife.82593.
17. Mengyu Li, Hongzhao Wang, Zhenwu Yang, Longgui Zhang, and Yushan Zhu. DeepTM: A deep learning algorithm for prediction of melting temperature of thermophilic proteins directly from sequences. *Computational and Structural Biotechnology Journal*, 21:5544–5560, 2023. doi: 10.1016/j.csbj.2023.11.006.
18. J. A. E. Alvarez and S. N. Dean. TEMPRO: nanobody melting temperature estimation model using protein embeddings. *Scientific Reports*, 14:19074, 2024. doi: 10.1038/s41598-024-70101-6.
19. Castrense Savojardo, Matteo Manfredi, Pier Luigi Martelli, and Rita Casadio. DDGemb: predicting protein stability change upon single- and multi-point variations with embeddings and deep learning. *Bioinformatics*, 41(1), 2024.
20. Aubin Ramon, Mingyang Ni, Olga Predeina, Rebecca Gaffey, Patrick Kunz, Shimobi Onuoha, and Pietro Sormanni. Prediction of protein biophysical traits from limited data: a case study on nanobody thermostability through NanoMelt. *MAbs*, 17(1):2442750, 2025.
21. Jun Mao, Yuanpeng Song, Ming Kong, Yanzhi Guo, Yijing Liu, and Xuemei Pu. Thermostability prediction powered by synergistic deep learning at experimental and theoretical levels for nanobodies. *ACS Appl. Mater. Interfaces*, 18(4):6433–6451, 2026.
22. Kotaro Tsuboyama, Justas Dauparas, Jonathan Chen, Elodie Laine, Yasser Mohseni Behbahani, Jonathan J. Weinstein, Niall M. Mangan, Sergey Ovchinnikov, and Gabriel J. Rocklin. Mega-scale experimental analysis of protein folding stability in biology and design. *Nature*, 620(7973):434–444, 2023. doi: 10.1038/s41586-023-06328-6.
23. Simon K. S. Chu, Kush Narang, and Justin B. Siegel. Protein stability prediction by fine-tuning a protein language model on a mega-scale dataset. *PLOS Computational Biology*, 20(7):e1012248, 2024. doi: 10.1371/journal.pcbi.1012248.
24. Mario S. Valdés-Tresanco, Mario E. Valdés-Tresanco, Esteban Molina-Abad, and Ernesto Moreno. NbThermo: a new thermostability database for nanobodies. *Database*, 2023: baad021, 2023. doi: 10.1093/database/baad021.
25. Yiming Zhang and Koji Tsuda. NbBench: Benchmarking language models for comprehensive nanobody tasks, 2025.
26. Robert Schmirler, Michael Heinzinger, and Burkhard Rost. Fine-tuning protein language models boosts predictions across diverse tasks. *Nature Communications*, 15:7407, 2024. doi: 10.1038/s41467-024-51844-2.
27. James Dunbar, Konrad Krawczyk, Jinwoo Leem, Terry Baker, Angelika Fuchs, Guy Georges, Jiye Shi, and Charlotte M. Deane. SAbDab: the structural antibody database. *Nucleic Acids Research*, 42(D1):D1140–D1146, 2014. doi: 10.1093/nar/gkt1043.
28. Todd J. Dolinsky, Jens E. Nielsen, J. Andrew McCammon, and Nathan A. Baker. PDB2PQR: an automated pipeline for the setup of Poisson–Boltzmann electrostatics calculations. *Nucleic Acids Research*, 32(Web Server issue):W665–W667, 2004. doi: 10.1093/nar/gkh381.
29. Mats H. M. Olsson, Chresten R. Søndergaard, Michal Rostkowski, and Jan H. Jensen. PROPKA3: consistent treatment of internal and surface residues in empirical pK_α predictions. *Journal of Chemical Theory and Computation*, 7(2):525–537, 2011. doi: 10.1021/ct100578z.
30. David A. Case, Hasan Metin Aktulga, Kellon Belfon, David S. Cerutti, G. Andres Cisneros, Vinicius W. D. Cruzeiro, Negin Forouzesh, Timothy J. Giese, Andreas W. Götz, Holger Gohlke, Saeed Izadi, Koushik Kasavajhala, Mehmet C. Kaymak, Edward King, Tom Kurtzman, Tai-Sung Lee, Pengfei Li, Jian Liu, Tyler Luchko, Ray Luo, Madushanka Manathunga, Matias R. Machado, Hai Minh Nguyen, Kurt A. O'Hearn, Alexey V. Onufriev, Feng Pan, Sergio Pantano, Ruxi Qi, Ali Rahnamoun, Ali Risheh, Stephan Schott-Verdugo, Akhil Shajan, Jason M. Swails, Junmei Wang, Haixin Wei, Xiongwu Wu, Yongxian Wu, Shi Zhang, Shiji Zhao, Qiang Zhu, Thomas E. Cheatham, Daniel R. Roe, Adrian E. Roitberg, Carlos Simmerling, Darrin M. York, Maria C. Nagan, and Kenneth M. Merz. AmberTools. *Journal of Chemical Information and Modeling*, 63(20):6183–6191, 2023. doi: 10.1021/acs.jcim.3c01153.
31. Chuan Tian, Koushik Kasavajhala, Kellon A. A. Belfon, Lauren Raguette, He Huang, Angela N. Migues, John Bickel, Yuzhang Wang, Jorge Pincay, Qin Wu, and Carlos Simmerling. ff19SB: amino-acid-specific protein backbone parameters trained against quantum mechanics energy surfaces in solution. *Journal of Chemical Theory and Computation*, 16 (1):528–552, 2020. doi: 10.1021/acs.jctc.9b00591.
32. Saeed Izadi, Ramu Anandakrishnan, and Alexey V. Onufriev. Building water models: a different approach. *The Journal of Physical Chemistry Letters*, 5(21):3863–3871, 2014. doi: 10.1021/jz501780a.
33. Peter Eastman, Jason Swails, John D. Chodera, Robert T. McGibbon, Yutong Zhao, Kyle A. Beauchamp, Lee-Ping Wang, Andrew C. Simmonett, Matthew P. Harrigan, Chaya D. Stern, Rafal P. Wiewiora, Bernard R. Brooks, and Vijay S. Pande. OpenMM 7: rapid development of high performance algorithms for molecular dynamics. *PLOS Computational Biology*, 13 (7):e1005659, 2017. doi: 10.1371/journal.pcbi.1005659.
34. Robert T. McGibbon, Kyle A. Beauchamp, Matthew P. Harrigan, Christoph Klein, Jason M. Swails, Carlos X. Hernández, Christian R. Schwantes, Lee-Ping Wang, Thomas J. Lane, and Vijay S. Pande. MDTraj: a modern open library for the analysis of molecular dynamics trajectories. *Biophysical Journal*, 109(8):1528–1532, 2015. doi: 10.1016/j.bpj.2015.08.015.
35. Robert B. Best, Gerhard Hummer, and William A. Eaton. Native contacts determine protein folding mechanisms in atomistic simulations. *Proceedings of the National Academy of Sciences of the United States of America*, 110(44):17874–17879, 2013. doi: 10.1073/pnas.1311599110.
36. James C. Phillips, David J. Hardy, Julio D. C. Maia, John E. Stone, João V. Ribeiro, Rafael C. Bernardi, Ronak Buch, Giacomo Fiorin, Jérôme Hénin, Wei Jiang, Ryan Mc-

Greevy, Marcelo C. R. Melo, Brian K. Radak, Robert D. Skeel, Abhishek Singharoy, Yi Wang, Benoît Roux, Aleksei Aksimentiev, Zaida Luthey-Schulten, Laxmikant V. Kalé, Klaus Schulten, Christophe Chipot, and Emad Tajkhorshid. Scalable molecular dynamics on CPU and GPU architectures with NAMD. *The Journal of Chemical Physics*, 153(4):044130, 2020. doi: 10.1063/5.0014475.

37. William Humphrey, Andrew Dalke, and Klaus Schulten. VMD: Visual molecular dynamics. *Journal of Molecular Graphics*, 14(1):33–38, 1996. doi: 10.1016/0263-7855(96)00018-5.
38. Vijayaraj Ramadoss, François Dehez, and Christophe Chipot. AlaScan: A graphical user interface for alanine scanning free-energy calculations. *Journal of Chemical Information and Modeling*, 56(6):1122–1126, 2016. doi: 10.1021/acs.jcim.6b00162.
39. Josh Abramson, Jonas Adler, Jack Dunger, Richard Evans, Tim Green, Alexander Pritzel, Olaf Ronneberger, Lindsay Willmore, Andrew J. Ballard, Joshua Bambrick, Sebastian W. Bodenstein, David A. Evans, Chia-Chun Hung, Michael O'Neill, David Reiman, Kathryn Tunyasuvunakool, Zachary Wu, Akvilė Žemgulytė, Eirini Arvaniti, Charles Beattie, Ottavia Bertolli, Alex Bridgland, Alexey Cherepanov, Miles Congreve, Alexander I. Cowen-Rivers, Andrew Cowie, Michael Figurnov, Fabian B. Fuchs, Hannah Gladman, Rishub Jain, Yousuf A. Khan, Caroline M. R. Low, Kuba Perlin, Anna Potapenko, Pascal Savy, Sukhdeep Singh, Adrian Stecula, Ashok Thillaisundaram, Catherine Tong, Sergei Yakneen, Ellen D. Zhong, Michal Zielinski, Augustin Žídek, Victor Bapst, Pushmeet Kohli, Max Jaderberg, Demis Hassabis, and John M. Jumper. Accurate structure prediction of biomolecular interactions with AlphaFold 3. *Nature*, 630(8016):493–500, 2024. doi: 10.1038/s41586-024-07487-w.
40. Robert B. Best, Xiao Zhu, Jihyun Shim, Pedro E. M. Lopes, Jeetain Mittal, Michael Feig, and Alexander D. MacKerell. Optimization of the additive CHARMM all-atom protein force field targeting improved sampling of the backbone ϕ, ψ and side-chain χ_1 and χ_2 dihedral angles. *Journal of Chemical Theory and Computation*, 8(9):3257–3273, 2012. doi: 10.1021/ct300400x.
41. William L. Jorgensen, Jayaraman Chandrasekhar, Jeffry D. Madura, Roger W. Impey, and Michael L. Klein. Comparison of simple potential functions for simulating liquid water. *The Journal of Chemical Physics*, 79(2):926–935, 1983. doi: 10.1063/1.445869.
42. Jean-Paul Ryckaert, Giovanni Ciccotti, and Herman J. C. Berendsen. Numerical integration of the cartesian equations of motion of a system with constraints: molecular dynamics of n-alkanes. *Journal of Computational Physics*, 23(3):327–341, 1977. doi: 10.1016/0021-9991(77)90098-5.
43. Ulrich Essmann, Lalith Perera, Max L. Berkowitz, Tom Darden, Hsing Lee, and Lee G. Pedersen. A smooth particle mesh Ewald method. *The Journal of Chemical Physics*, 103 (19):8577–8593, 1995. doi: 10.1063/1.470117.
44. Charles H. Bennett. Efficient estimation of free energy differences from Monte Carlo data. *Journal of Computational Physics*, 22(2):245–268, 1976. doi: 10.1016/0021-9991(76)90078-4.
45. Elizabeth H. Kellogg, Andrew Leaver-Fay, and David Baker. Role of conformational sampling in computing mutation-induced changes in protein structure and stability. *Proteins: Structure, Function, and Bioinformatics*, 79(3):830–838, 2011. doi: 10.1002/prot.22921.
46. Rebecca F. Alford, Andrew Leaver-Fay, Jeliazko R. Jeliazkov, Matthew J. O'Meara, Frank P. DiMaio, Hahnbeom Park, Maxim V. Shapovalov, P. Douglas Renfrew, Vikram K. Mulligan, Kalli Kappel, Jason W. Labonte, Michael S. Pacella, Richard Bonneau, Philip Bradley, Roland L. Dunbrack, Rhiju Das, David Baker, Brian Kuhlman, Tanja Kortemme, and Jeffrey J. Gray. The Rosetta all-atom energy function for macromolecular modeling and design. *Journal of Chemical Theory and Computation*, 13(6):3031–3048, 2017. doi: 10.1021/acs.jctc.7b00125.
47. Henry Dieckhaus, Michael Brocidiacono, Nicholas Z. Randolph, and Brian Kuhlman. Transfer learning to leverage larger datasets for improved prediction of protein stability changes. *Proceedings of the National Academy of Sciences of the United States of America*, 121(6):e2314853121, 2024. doi: 10.1073/pnas.2314853121.
48. Alex Kendall, Yarin Gal, and Roberto Cipolla. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 7482–7491, 2018. doi: 10.1109/CVPR.2018.00781.

Supplementary Methods

Experimental target data and target-label scaling

All target-task comparisons use the same NbBench thermo-seq endpoint (25). The processed tables hold an amino-acid sequence field and a melting-temperature label in degrees Celsius. The split sizes are 57 training, 114 validation, and 396 held-out test sequences. As described in Methods, we deliberately assigned the smallest NbBench partition to training and the largest to the test set so that the model sits in the low-data regime the study targets.

Split definitions. The splits are fixed throughout. The 57 training examples fit network parameters, the validation split drives checkpoint selection, early stopping, and choice among candidate settings, and the test split stays untouched until those choices are fixed. The same rule applies to Tm-only models and to models trained with computational labels.

Target-label scaling. Before optimization, the training workflow maps the experimental Tm values to a scaled regression target. The scaler is fit only on the 57 training labels, using robust scaling followed by linear rescaling to [0,1]. Validation labels are transformed with this training-fitted scaler during model selection. Test predictions are averaged across seeds in scaled-label space and inverse-transformed to degrees Celsius before errors are computed, so the validation and test label distributions never influence the fitted transform.

Experimental-label scaling reference. This reference uses the same training split but subsamples it to the requested number of labels with the run seed; the validation and test splits are never subsampled. It serves only as a positive control for the pipeline. Adding true target labels should lower the Tm prediction error.

Processed computational-label tables

Computational labels are loaded as computational tasks rather than appended to the Tm data as extra measurements. Table 1 lists the processed target and computational-label files used in the reported comparisons. Mutation-effect tables carry sequence fields and [0,1]-scaled computational-label fields. The MD-derived Q-value table uses the same processed format so that it can enter the same computational-task loader; there, the scaled label is a [0,1]-scaled Q-value rather than a mutation free energy.

Table 1. Processed target and computational-label data. Row counts exclude header rows.

Data type	Role	Processed rows	Use in reported comparisons
Experimental Tm	Target train	57	Model fitting
Experimental Tm	Target validation	114	Checkpoint and setting selection
Experimental Tm	Target test	396	Final held-out evaluation
FEP mutation free energy	Computational tasks	435 + 409	Main computational task
Rosetta mutation score	Computational tasks	435 + 409	Computational-label comparison
ThermoMPNN stability score	Computational tasks	435 + 409	Computational-label comparison
Random variants scored by Rosetta	Computational tasks	1000 + 1000	Computational-label comparison
ESM2-proposed variants scored by Rosetta	Computational tasks	1000 + 1000	Computational-label comparison
MD-derived Q-value	Computational task	1143	Computational-label comparison and scaling control

Computational-task assignment. The loader assigns each row to a target or computational task. Experimental Tm rows use the Tm head. In the main mutation-effect model, the two template-specific tables use separate computational heads, and the MD-derived Q-value table uses a single computational head. This assignment fixes which regression head an example uses and which task-specific loss term it contributes to.

Sequence-overlap check. We checked exact sequence overlap between the processed computational tables and the fixed NbBench splits. The FEP/Rosetta mutation-effect tables, the random variants scored by Rosetta, and the ESM2-proposed variants scored by Rosetta had no exact match to any NbBench training, validation, or test sequence. The ThermoMPNN tables used the same template-specific variant design as the mutation-effect computational labels. The SAbDab-derived MD Q-value table contained exact matches to 2 NbBench training, 4 validation, and 8 test sequences. We disclose this because it bears on interpretation of the MD control; it cannot explain the positive FEP result, and the MD Q-value computational label did not improve held-out Tm prediction.

Computational-label sampling. For mutation-effect experiments with requested count n, the loader samples up to n rows independently from each of the two template tables using the run seed; the final FEP setting with $n = 320$ therefore supplies 320 sampled 1MEL rows and 320 sampled 4IDL rows before the computational-task split. For MD-derived Q-value experiments, the loader samples n rows from the single Q-value table. Computational-label rows are split 80/20 into computational-training and computational-validation rows with the same seed. The production training call then filters the validation data passed to the trainer so that checkpoint selection and early stopping depend only on the experimental Tm validation examples.

SAbDab-derived nanobody structure panel for MD

The MD computational-label panel was built from SAbDab, the Structural Antibody Database (27). The archived SAbDab nanobody summary table holds 2422 rows with PDB identifier, heavy- and light-chain assignments, model index, antigen annotation, experimental method, resolution, species, and other fields. The manifest workflow matches this summary against the archived IMGT-renumbered PDB files.

Structure-selection rule. The rule is deterministic. For each PDB identifier with both a SAbDab summary row and an IMGT-renumbered PDB file, the workflow takes the first heavy-chain row in the SAbDab table and accepts the chain only if that chain identifier is present in the IMGT PDB file. The output manifest holds one entry per PDB identifier, with fields for the manifest row number, PDB identifier, selected heavy chain, SAbDab TSV row, IMGT PDB path, and MD run directory. This initial one-per-PDB manifest contained 1177 structures.

400 K simulation panel. The 400 K campaign reused the systems prepared in the 300 K campaign and built a 1146-entry 400 K manifest, consisting of 798 X-ray, 345 electron-microscopy, and 3 NMR structures according to the SAbDab method field. None of the selected rows had an annotated light chain, consistent with a single-domain antibody panel. After trajectory and Q-value processing, 1143 unique PDB identifiers yielded usable computational-label rows.

MD system preparation and simulation protocol

System preparation. Each structure was prepared as a nanobody-only system. The workflow takes the IMGT-renumbered PDB as the input structure, extracts the selected heavy chain, keeps only protein atoms, and leaves the termini uncapped, at pH 7.4. Chains are cleaned with PDBFixer, and pH-aware protonation is assigned through pdb2pqr/PROPKA when available (28, 29), with a fallback to AmberTools-based hydrogen assignment and Amber naming. The prepared chain is solvated in explicit OPC water with 15 Å padding and 0.15 M NaCl, and Amber topology and coordinate files are built with ff19SB and OPC (30–32).

300 K equilibration branch. The base 300 K branch uses OpenMM on CUDA devices (33). It runs 1 ns of NVT followed by 1 ns of NPT at 300 K and 1 atm, with a 2-fs timestep and no hydrogen-mass repartitioning. Production at 300 K was generated for the broader campaign, but the computational label used here comes from the 400 K branch.

400 K production branch. The first 400 K relaxation stage restarts from the 300 K equilibrated state and runs 1 ns of NVT at 400 K with Cα restraints; the second repeats 1 ns of restrained 400 K NVT. Both relaxation stages use a 2-fs timestep and no hydrogen-mass repartitioning. Production runs 100 ns of restraint-free 400 K NVT, with hydrogen-mass repartitioning, 4 amu hydrogen mass, a 4-fs timestep, and coordinate/energy output every 100 ps.

Integrator and reporting settings. In OpenMM, explicit-solvent periodic systems use PME electrostatics with a 1.0-nm nonbonded cutoff and constraints on bonds to hydrogen. NPT stages use a Monte Carlo barostat; NVT stages omit it. The integrator is the Langevin middle integrator with 1 ps^{-1} friction. The 100-ps reporting interval gives 1000 expected frames for a complete 100 ns production trajectory.

MD-derived Q-value extraction

Trajectory input requirements. The processed MD table was generated from the archived 400 K production trajectories. A system was eligible only if it had a production trajectory, a prepared reference structure, and an Amber parameter file. The amino-acid sequence was read from the prepared structure using the first Cα atom of each residue, and sequences shorter than 50 residues were excluded.

Native-contact definition. Q-values are computed with MDTraj (34). The topology is loaded from the Amber parameter file, and the native reference is frame 0 of production. The calculation uses protein backbone heavy atoms. A candidate native contact must span more than three residues, have a reference-frame distance below 0.45 nm, and include at least one atom from a hydrophilic residue (Asp, Glu, Gln, Asn, Arg, Lys, or His, including Amber protonation variants). For a native-contact pair (i,j) with reference distance r_{ij}^0 and instantaneous distance r_{ij}, the per-frame contribution is

$$q_{ij} = \frac{1}{1+\exp\{\beta(r_{ij}-\lambda r_{ij}^0)\}},$$

with $\beta = 50\ \mathrm{nm}^{-1}$ and $\lambda = 1.8$, following the smooth native-contact form of Best-Hummer-style Q-value analyses (35). The frame-level Q-value is the mean of q_{ij} over all selected native contacts.

Trajectory-level Q-value label. The trajectory-level label is the mean Q-value over the terminal portion of production. For trajectories with more than 300 frames, the implementation uses the larger of 300 frames and one third of the available frames; for shorter trajectories it uses all frames. For a complete 100 ns trajectory with 100-ps output, this is 333 terminal frames, roughly the final 33 ns. In the processed 400 K table, the number of frames used ranges from 66 to 333 (median 333), the number of native contacts from 28 to 728 (median 173), and the sequence length from 58 to 461 residues (median 122). Raw Q-values range from 0.0437 to 0.9994 and are scaled to [0,1] for computational-task training.

Sequence tokenization and encoder inputs

Sequences are tokenized with the ESM2 tokenizer for the selected encoder (14), using truncation and fixed-length padding to 160 residues. In the production protocol, the trainer receives all target and sampled computational-label rows for training, but its evaluation data are filtered to the experimental Tm validation rows.

Hot-encoder input path. The hot-encoder setting keeps tokenized sequences in the training data and updates the ESM2 weights. The frozen-encoder setting runs ESM2 once per target and computational-label sequence to precompute first-token embeddings, after which the optimizer updates only the regression trunk and task heads.

Neural-network architecture

The default encoder is the 8M-parameter ESM2 model, and the model uses its first-token representation as the sequence-level embedding. The shared trunk has dimensions hidden-size $\rightarrow$ 256 $\rightarrow$ 128 $\rightarrow$ 32, with ReLU activations and dropout after the first two hidden layers. The Tm head maps the 32-dimensional representation to one scaled scalar prediction.

Target and computational heads. The Tm-only baseline is the encoder, the shared trunk, and the Tm head. Computational-label transfer models keep this target path and add computational heads. The main mutation-effect model uses separate computational heads for the two template-specific tasks. The code also implements computational-head controls, including a single shared mutation-effect head, a template-conditioned head, and a template-specific affine calibration on top of a shared head, to test whether the two template-specific computational labels need separate output calibration.

Architecture sensitivity analyses. Architecture controls include residual, latent, and mixture-style computational-fusion variants, in which the computational path produces a correction to the Tm prediction, a latent interaction between Tm and computational features, or a gate between a target-only expert and a computational-informed expert. The main reported FEP result uses the original shared-trunk architecture with separate template-specific computational heads; the other variants are kept as sensitivity analyses, not as the main claim.

Training objective, optimizer, and checkpoint selection

Each training example is a sequence, a scaled label, and a task identifier, and contributes loss only through the regression head for its task. The per-task loss is Huber loss with $\delta = 1.0$ on the scaled label. The default multi-task objective uses learned task-uncertainty weights (48). For task k with loss L_k and learned log-scale s_k, the contribution is

$$\frac{L_k}{2\exp(2s_k)} + s_k,$$

and the total loss sums the contributions from the tasks present in the minibatch. Fixed computational-loss weights are also implemented and were included in the candidate searches for computational classes where loss balance could shift validation performance.

Optimizer and schedule. The optimizer is AdamW. The default head/trunk learning rate is 3×10^{-4}, the default hot-encoder learning rate is 1×10^{-4}, weight decay is 0.04, dropout is 0.15, batch size is 16, warmup is 100 steps, and the schedule is cosine decay over at most 400 epochs. Checkpoints are evaluated and saved once per epoch, and mixed precision is used when CUDA is available. In the hot-encoder setting, the trainer creates separate optimizer parameter groups for ESM2 parameters and for the freshly initialized trunk/head parameters; in the frozen-encoder setting, ESM2 parameters are non-trainable and no encoder group is used.

Checkpoint selection. Within each run, the best checkpoint has the lowest mean squared error on the experimental Tm validation rows, and early stopping monitors the same metric with patience 15 and threshold 0. Computational-task validation rows are not passed to the trainer in the production protocol. This detail is central to the comparison. Computational labels shape the learned representation through the training loss, but they do not decide which checkpoint is used for target-task prediction.

Candidate-setting selection and final evaluation

Candidate settings are selected on the experimental Tm validation split. The common grid includes the default setting; encoder learning rates of 3×10^{-5}, 1×10^{-4}, and 3×10^{-4}; a lower head learning rate of 1×10^{-4} paired with encoder learning rate 3×10^{-5}; and dropout rates of 0.05, 0.15, and 0.30. Fixed computational-loss weights are added for computational classes where the computational-label scale may need explicit balancing. Tm-only candidates include shared, residual, and latent architecture variants; mutation-effect candidates use the shared-trunk family for the main comparison, with a separate computational-head design comparison; and MD-derived candidates include fixed-weight settings because Q-value labels differ in scale and interpretation from mutation free energies.

Seeded candidate evaluation. Each candidate is first evaluated as a three-seed ensemble on the Tm validation split, and the selected setting for each computational class is then evaluated on the held-out test split as a ten-seed ensemble. Final comparisons use seeds 1 to 10. The run seed controls Python, NumPy, PyTorch, and CUDA randomness when CUDA is available,

as well as target-training subsampling in the target-label scaling experiment, computational-row sampling, and computational-label train/validation splitting. The label-count curves use fixed computational-specific training recipes while the number of computational labels is varied.

Statistics and stored outputs

The primary metric is MAE in degrees Celsius on the 396 held-out test examples. For each condition, the ten seeded models produce scaled Tm predictions for every test sequence; these are averaged in scaled-label space and inverse-transformed to degrees Celsius. Root mean squared error, coefficient of determination, and Spearman and Pearson correlations are also computed, but the figures emphasize MAE because it reads directly as an average temperature error.

Bootstrap intervals. Single-condition confidence intervals come from nonparametric bootstrap resampling over test examples. The absolute-error vector from the ten-seed ensemble is resampled with replacement 10,000 times using a fixed bootstrap seed, and the reported 90% interval is the 5th to 95th percentile range of the bootstrapped MAE values. The 90% interval is used for consistency with the directional paired comparisons in the main text; the stored bootstrap summaries allow other interval levels to be recomputed. Paired comparisons use the same bootstrap indices for the candidate and reference absolute-error vectors. The reported ΔMAE is candidate minus reference, so negative values mean lower error than the reference.

Stored run summaries. Each training and evaluation call writes a structured summary recording the command-line arguments, relevant environment variables, resolved model settings, per-label-count metrics, bootstrap intervals, per-example absolute errors, and paired bootstrap summaries. Compact tables derived from these summaries feed the main computational-label comparison, the frozen-encoder controls, the computational-head controls, and the computational-combination controls.

Interpreting the label-count curves

Target-label scaling reference. The target-label scaling curve is a positive control on the pipeline. It tests whether the same model family and evaluation code recover the expected gain when more true experimental Tm labels are available. The FEP computational-label curve tests something different. It asks whether more labels from a physically related but distinct computed quantity improve the same held-out Tm endpoint, while the MD Q-value curve tests whether simply adding more simulation-derived labels suffices when the computational label measures a more indirect structural response. The decisive comparison is therefore not "simulation versus no simulation," but whether the computational label carries physical information that transfers to the target property under target-centered model selection.

Supplementary figure data tables

The supplementary figures are generated from tabular computational-label files derived from the same run summaries as the main figures. The processed computational tables, rendered supplementary figures, and figure-generation code will be distributed with the public analysis archive.

Fig.-to-data and code mapping

The manuscript figures are regenerated from compact result summaries and computational-label tables rather than from scratch run directories. The mapping below is the first place to check when auditing or extending a figure.

Table 2. Manuscript Fig.-to-data mapping. The table identifies the processed summaries and scripts used to regenerate each display item.

Display item	Inputs	Script	Primary comparison
Fig. 1	Conceptual protocol	`plot/make_outline_figures.py`	Target-centered transfer-learning design
Fig. 2	Main count-sweep summaries	`plot/make_outline_figures.py`	Label-count sweeps for Tm labels, FEP labels, and MD Q-value labels
Fig. 3	Final hot- and frozen-encoder computational-label screen summaries	`plot/make_outline_figures.py`	All-atom FEP and MD labels versus training only on experimental Tm labels
Fig. 4	Final computational-label screen summary	`plot/make_outline_figures.py`	Rosetta, ThermoMPNN, and Rosetta-scored proposal sets
Supplementary Figs. 1 to 5	Supplementary manifest and generated TSV tables	`plot/make_supplementary_figures.py`	Data-set composition, candidate-setting searches, model controls, count-sweep controls, and MD-feature controls

Generated supplementary tables. The final computational-label screen settings used in Figs. 3 and 4 and the frozen-encoder controls are recorded in generated supplementary tables, which list the selected architecture, validation-selected setting, computational-label count, validation MAE, held-out test MAE, bootstrap interval, and run-summary reference for each condition. A panel-level mapping table for all supplementary figures is included in the public analysis archive.

Supplementary Figs.

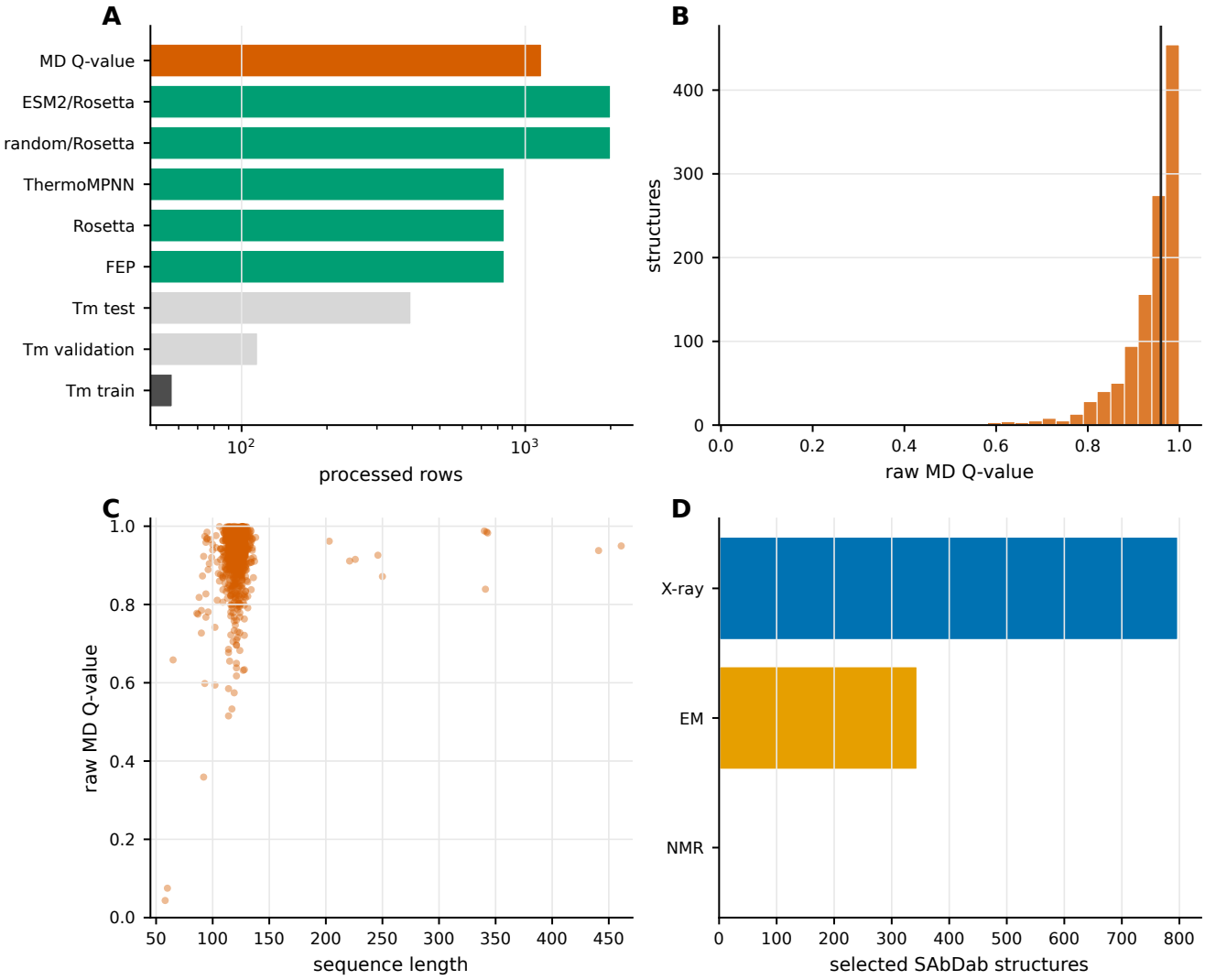

Fig. 5. Supplementary Fig. 1. Processed data sets and MD-derived Q-value labels. (a) Processed target and computational-label row counts used in the reported experiments. **(b)** Distribution of raw MD-derived Q-values before [0,1] scaling. **(c)** Relationship between sequence length and raw MD-derived Q-value. **(d)** Experimental methods for the selected SAbDab structures used in the 400 K MD panel.

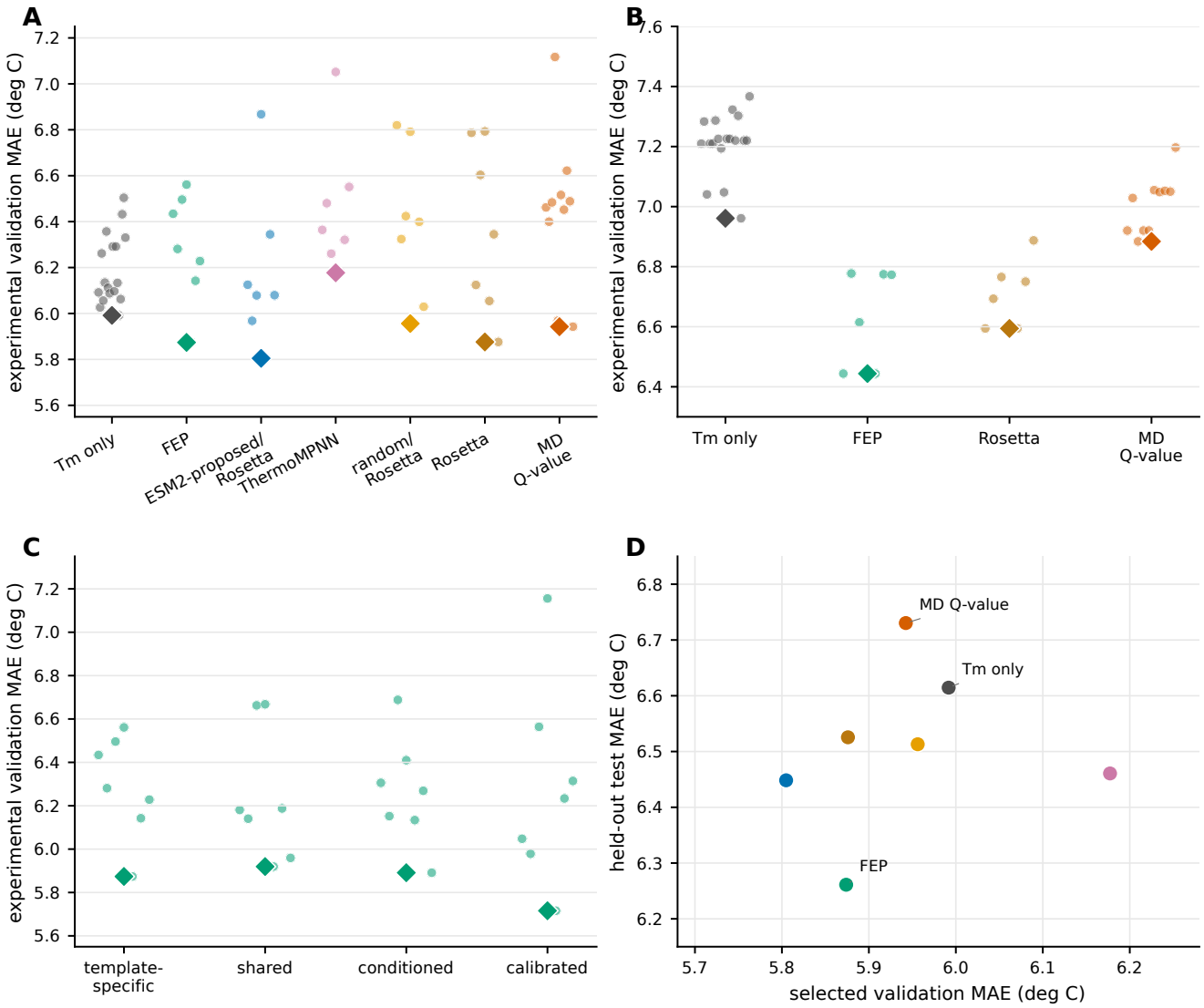

Fig. 6. Supplementary Fig. 2. Candidate-setting searches and target-validation selection. **(a)** Experimental validation MAE values for hot-encoder computational-label comparisons. **(b)** Experimental validation MAE values for frozen-encoder controls. **(c)** Candidate computational-head designs for FEP mutation free-energy labels. **(d)** Relationship between selected validation MAE and held-out test MAE for the final computational-label comparison. Diamonds mark the best validation setting in panels a to c.

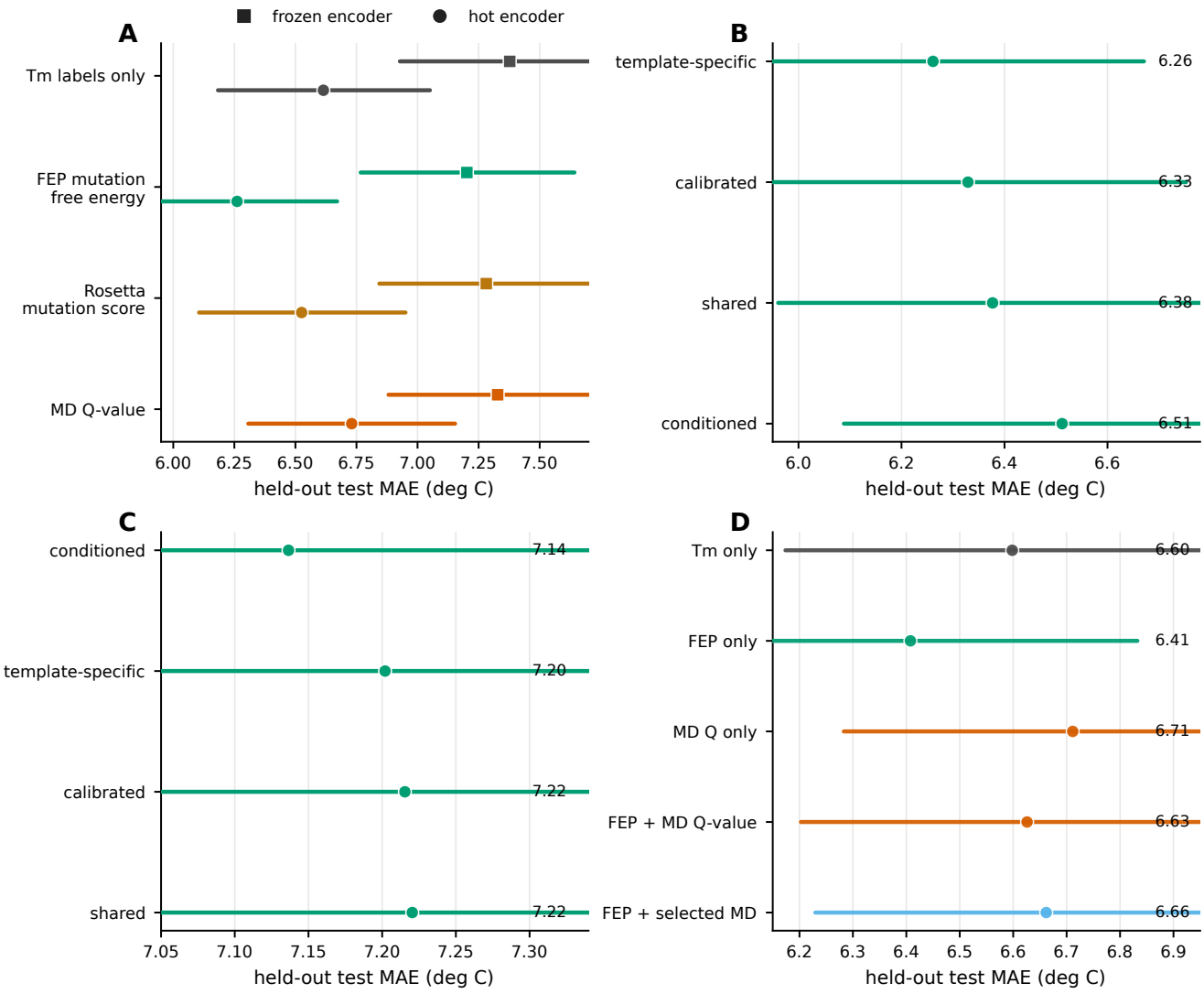

Fig. 7. Supplementary Fig. 3. Model and computational-combination controls. (a) Frozen-encoder and hot-encoder comparisons for selected computational labels. **(b)** Hot-encoder FEP computational-head controls. **(c)** Frozen-encoder FEP computational-head controls. **(d)** Computational-combination controls comparing Tm-only, FEP-only, MD Q-value-only, and combined-computational-label settings.

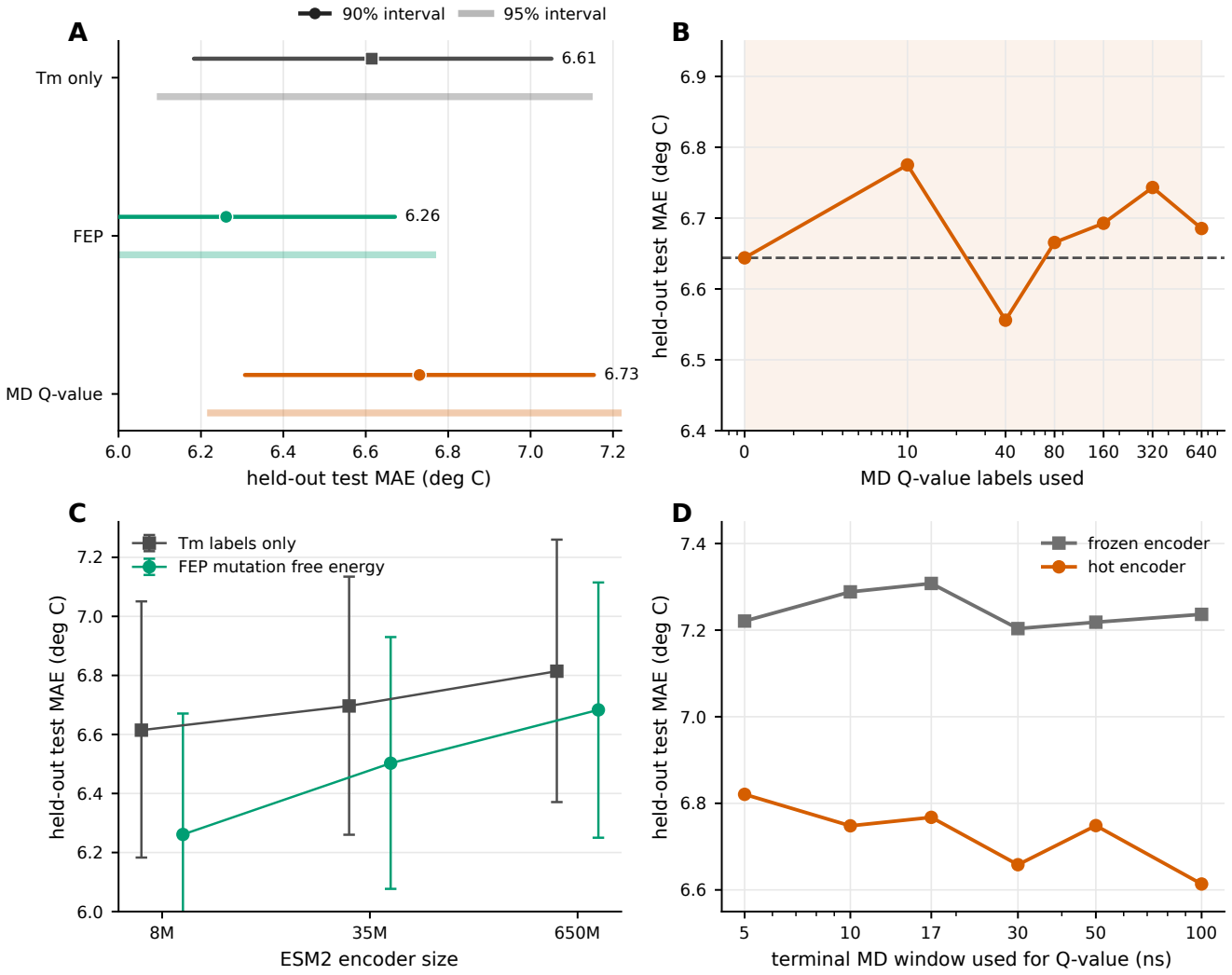

Fig. 8. Supplementary Fig. 4. Interval, scaling, encoder-size, and MD-window controls. **(a)** Final Tm-only, FEP, and MD Q-value conditions shown with 90% and 95% bootstrap intervals. **(b)** MD Q-value label-count curve after selecting the candidate setting separately for each label count on the experimental validation split. **(c)** ESM2 encoder-size controls for Tm-only and FEP settings. **(d)** MD trajectory-window controls for Q-value labels.

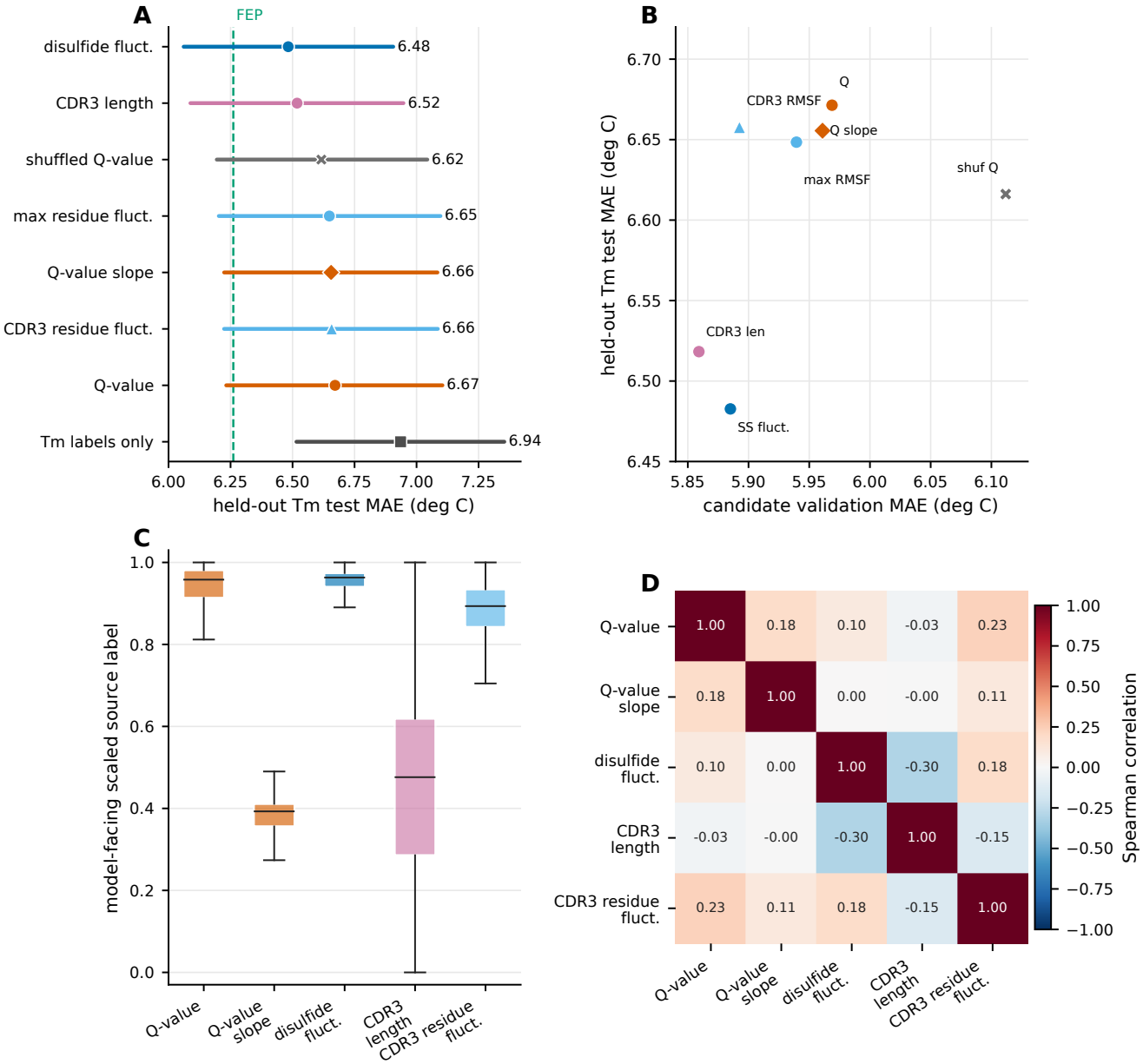

Fig. 9. Supplementary Fig. 5. Descriptor controls for MD-derived computational labels. (a) Held-out test MAE values for selected MD-derived and nanobody-specific descriptor controls. The dashed line marks the FEP reference. **(b)** Relationship between candidate validation MAE and held-out test MAE for descriptor controls that were carried forward to final evaluation. **(c)** Distributions of the model-facing scaled computational labels for the selected descriptors. **(d)** Spearman correlations among the selected computational labels over shared sequences.